Ensino que te conecta

BACHARELADO EM

ENGENHARIA DE SOFTWARE

ACCELERATION OF PATTERN MATCHING ALGORITHMS USING PARALLEL PROGRAMMING

MATHEUS ANTÔNIO DE CASTRO DE BARROS

Brasília - DF, 2025

MATHEUS ANTÔNIO DE CASTRO DE BARROS

ACCELERATION OF PATTERN MATCHING ALGORITHMS USING PARALLEL PROGRAMMING

Qualification Exam document submitted as a partial requirement for the Bachelor's Degree in Software Engineering at the Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa (IDP).

Orientador
Jeremias Moreira Gomes

Brasília - DF, 2025

MATHEUS ANTÔNIO DE CASTRO DE BARROS

ACCELERATION OF PATTERN MATCHING ALGORITHMS USING PARALLEL PROGRAMMING

Qualification Exam document submitted as a partial requirement for the Bachelor's Degree in Software Engineering at the Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa (IDP).

Aprovado em 08/05/2025

Banca Examinadora

Patrícia Oliveira

Lucas Maurício

Jeremias Moreira Gomes

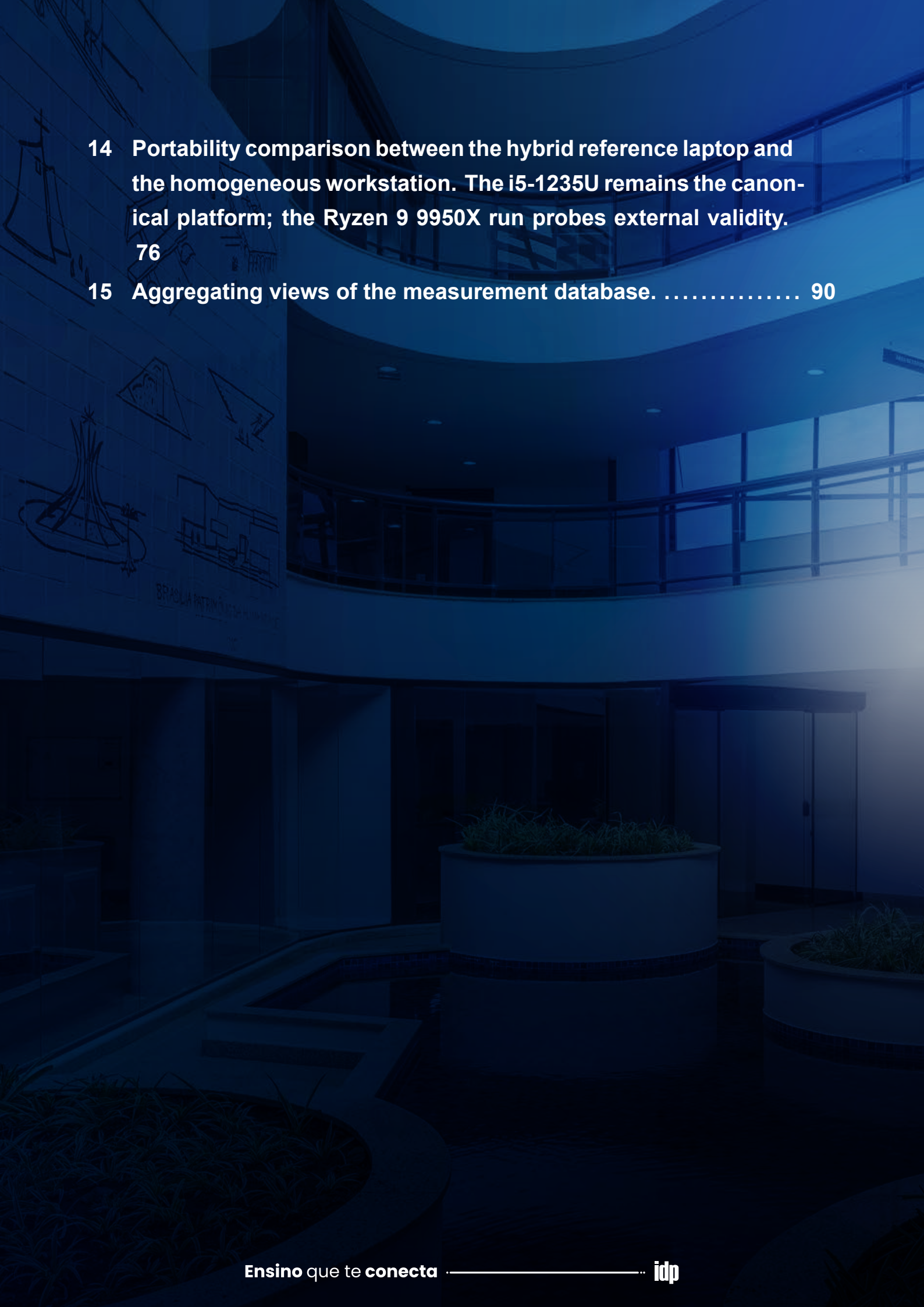
ABSTRACT

The Aho-Corasick algorithm is a cornerstone of signature-based network intrusion detection systems and malware analysis tools, because of its linear-time complexity in the size of the input. Its state-transition dependency, however, ties the canonical implementation to sequential execution, leaving modern multi-core processors underused. This work designs, implements, and evaluates data-parallel scanning strategies built on POSIX Threads for shared-memory multi-core CPUs. The input text is partitioned into overlapping segments that worker threads scan concurrently over a shared, strictly read-only automaton, keeping the hot path free of locks; a flattened, contiguous output layout is also proposed to eliminate the pointer-chasing cost of emitting matches. The study deliberately targets the memory-bound regime where the automaton greatly exceeds the last-level cache, using realistic Snort and Emerging Threats rule sets (up to roughly 507 MiB of automaton) over multi-gigabyte corpora on a hybrid Intel Core i5-1235U processor. The results show that text-level data parallelism outperforms pattern-set partitioning: search speedups reach up to $4.79\times$ on the canonical corpus and $4.52\times$ on the multi-gigabyte corpus — both at twelve threads for the same moderate dictionary — while the flattened layout adds a regime-dependent gain that grows with memory pressure, up to $+13.0$% on a single thread. When the automaton is forty-two times larger than the cache, however, throughput drops by 74.0% and parallel scaling collapses: on the canonical corpus the speedup peaks near $2.85\times$ at six threads and then regresses, while on the multi-gigabyte corpus it saturates at only $2.79\times$ at twelve threads, exposing memory bandwidth as the physical ceiling. Whole-pipeline (build, search, and merge) accelerations reach $4.50\times$ on the moderate dictionary and $2.76\times$ on the memory-bound one; since construction is under one percent of the runtime, these track the search-only speedups almost exactly. The central finding is that, once the automaton exceeds the last-level cache, the dominant barrier to scaling Aho-Corasick is microarchitectural — namely cache overflow and DRAM bandwidth — rather than the matching logic itself.

Keywords: Aho–Corasick, parallel programming, parallel computing, multi-pattern matching, POSIX threads, .

LIST OF FIGURES

LIST OF TABLES



CONTENTS


1

1 INTRODUCTION

Advancements in semiconductor technology have historically driven the growth of computational power. For decades, the number of transistors on a chip doubled roughly every eighteen months, an observation known as *Moore's Law* [1]; rising clock speeds translated these denser circuits into proportional single-core performance gains. However, in the mid-2000s, the hardware industry encountered significant physical barriers, such as excessive power consumption and challenges in heat dissipation, which limited the increase in processor clock frequency.

The response to these technological limitations was a change in processor design: instead of focusing on making a single core faster, the emphasis shifted to integrating multiple processing cores onto a single chip. Thus, multi-core processors emerged as the dominant architecture in modern computers. This transition opened up new possibilities for performance enhancement; however, to take advantage of multi-core processors, programs must be intentionally written or adapted to execute tasks in parallel [1].

This architectural shift from sequential to parallel execution has a profound impact on high-performance applications. In many scenarios, such as bioinformatics pipelines and network intrusion detection, processing large data volumes efficiently is essential. These applications rely on highly efficient multi-pattern-matching algorithms, and the Aho-Corasick algorithm stands out because it can search for many patterns simultaneously while scanning the input only once, achieving linear-time complexity with respect to the size of the input text. Widely adopted in security tools — including YARA for malware classification and Snort and Suricata for signature-based network intrusion detection [2] — the Aho-Corasick algorithm serves as a backbone for multi-pattern search engines [3].

Despite its efficiency, the Aho-Corasick algorithm is intrinsically sequential: its automaton processes input strictly left-to-right, with each state transition depending on the preceding one. As a result, when scanning a single large input on a modern multi-core CPU, the entire computational load is concentrated on one core. This serial dependency becomes a major performance bottleneck when scanning large target data, such as full disk images, packet captures, or genomic sequences. Consequently, the disparity between the algorithm's sequential nature and modern hardware's parallel

processing capabilities widens as data volumes grow [4].

1.1 MOTIVATION

The intrinsically sequential nature of Aho-Corasick, combined with the prevalence of multi-core architectures in general-purpose computing, presents a clear opportunity for acceleration through parallel computing. Previous studies report speedups on specialized platforms and in specific application domains [5], [2]; however, the behavior of a shared-memory, multi-threaded implementation on commodity CPUs in the *memory-bound regime* — where the automaton size far exceeds the last-level cache — has received comparatively little attention. This observation is supported by the systematic literature review presented in Chapter 3.

As pattern sets for tools such as Snort and Suricata grow in size, the corresponding Aho-Corasick automata can occupy hundreds of megabytes, pushing the hot data structure well beyond the cache hierarchy [6]. In this regime, the primary performance bottleneck shifts from computation to memory bandwidth [7], [2]. The interaction of multiple threads sharing the same last-level cache changes the scaling behavior in ways that compute-bound models do not capture. Large signature sets such as those of Snort and Suricata are one prominent example of how such automata arise in practice; understanding the limits of parallel scaling on these realistic workloads is the motivation for this study, with intrusion detection serving as a representative application rather than the object of the work.

1.2 OBJECTIVES

The general objective of this thesis is to design, implement, and evaluate a parallel version of the Aho-Corasick algorithm targeting shared-memory multi-core CPU architectures, using the POSIX Threads programming model, in order to accelerate multi-pattern matching on large inputs.

To achieve this general objective, the following specific objectives are defined:

1. To conduct a systematic literature review of parallelization and optimization techniques for the Aho-Corasick algorithm on multi-core CPUs, in order to identify the state of the art and delimit the research gap addressed by this work;

2. To design and implement a parallel version of the Aho-Corasick algorithm in C, using the pthreads API, exploiting intra-input parallelism on shared-memory multi-core CPUs;

3. To conduct a rigorous experimental evaluation that quantifies the performance gains of the parallel implementation in terms of speedup, throughput, and parallel efficiency, against a sequential baseline;

4. To analyze the scalability of the proposed solution under different workload, pattern-set, and thread-count configurations.

1.3 STRUCTURE OF THE THESIS

This document is organized as follows. Chapter 2 presents the Theoretical Framework, introducing the fundamentals of parallel computing, including architectures, performance metrics, and thread-based programming models, with emphasis on the POSIX Threads API. It also discusses the core data structures and algorithms relevant to this work, including Tries, the general pattern-matching problem, and a detailed analysis of the Aho-Corasick algorithm. Chapter 3 describes the Systematic Literature Review, presenting the research questions, search strategy, and selection criteria adopted to identify the state of the art and define the research gap. Chapter 4 presents the Proposed Solution, detailing the parallel-scanning strategy, the boundary-overlap and load-balancing mechanisms, and the flattened output layout that constitute the design. Chapter 5 describes the Methodology, specifying the experimental environment, datasets, metrics, measurement protocol, correctness validation, and the hypotheses that the experiments are designed to test. Chapter 6 reports and discusses the Results, isolating the contribution of each optimization and positioning the findings against the related literature. Finally, Chapter 7 presents the Conclusion, summarizing the contributions, the threats to validity, and directions for future work.

2

2 THEORETICAL FRAMEWORK

2.1 FUNDAMENTALS OF PARALLELISM

Parallelism in computing refers to the simultaneous execution of multiple tasks or parts of the same task, with the primary goal of reducing a program's total execution time. It is crucial to distinguish parallelism from concurrency. Concurrency occurs when multiple tasks are in progress at a given time, but may not be executing simultaneously. A single-core system can exhibit concurrency by rapidly switching between the execution of different tasks, simulating parallelism through preemption. True parallelism, on the other hand, requires hardware with multiple processing units [8]. This distinction is fundamental to the present work, which exploits true hardware parallelism through multiple CPU cores.

2.1.1 Parallel Computing Architectures

Parallel computer architectures are classified according to various criteria. A prominent taxonomy, proposed by Flynn [9], categorizes these architectures based on instruction and data streams:

- **SISD (Single Instruction, Single Data)**: Represents a conventional single-processor architecture, where a single instruction operates on a single data stream;
- **SIMD (Single Instruction, Multiple Data)**: Involves multiple processing units that execute the same instruction concurrently on different data elements;
- **MISD (Multiple Instruction, Single Data)**: Characterizes systems where multiple processing units apply distinct instructions on the same data stream;
- **MIMD (Multiple Instruction, Multiple Data)**: Involves multiple processing units that execute distinct instructions on distinct data streams independently. This paradigm is directly related to multi-core architectures.

Within the MIMD category, systems are further distinguished based on their memory organization [1]:

- **Shared-Memory Systems**: In these systems, multiple processors share a physical memory address space. Inter-processor communication is typically achieved by reading from and writing to shared variables in memory. Multi-core processors are the most common and widely available example of this architecture, where all cores on a chip share access to main memory;
- **Distributed-Memory Systems**: Each processor in these systems has its own locally accessible memory, and communication between processors is usually done by exchanging messages across an interconnection network. Computer clusters are examples of this architectural model.

The present work focuses on the acceleration of algorithms in environments with multi-core processors, which fall into the shared memory MIMD category. A multi-core processor (MCP) is a single chip that contains two or more central processing units (CPUs) called 'cores'. Each of these cores is capable of reading and executing program instructions independently, essentially functioning as a complete processor.

2.1.2 Processes vs Threads

Within shared-memory systems, such as multi-core processors, software-level parallelism is commonly achieved through the execution of multiple threads within a single process.

A process is an instance of an executing program, an entity possessing its own dedicated virtual address space, file descriptors, environment variables, and other Operating System (OS) resources, all isolated from other processes [1].

A thread, on the other hand, is a lighter unit of execution started by a process. The same process can control, concurrently or in parallel, multiple threads. A key characteristic is that all threads within a process share its virtual address space, which encompasses executable code, global data, and the heap segment [1]. However, each thread maintains its private resources, which are essential for its independent execution context:

- A unique identifier (thread ID);
- A set of CPU registers, including the program counter (PC);
- A separate execution stack, used for local variables and function call control;
- Thread-specific state, such as signal masking and scheduling priority.

2.1.3 POSIX Thread Model (Pthreads)

The POSIX Thread Model, commonly referred to as "Pthreads", is a standard that defines an application programming interface (API) for thread creation and management, as specified by IEEE Std 1003.1c-1995. Pthreads provides a portable suite of

data types, functions, and constants that allows developers to create multithreaded applications that are compilable and executable across diverse POSIX-compliant operating systems (e.g., Linux, macOS, and various Unix variants). The Pthreads API provides functionality for managing the thread lifecycle—including creation, termination, and joining—and, critically, incorporates synchronization primitives. These primitives, such as mutexes and condition variables, are essential for managing access to shared resources within shared-memory environments [8] [1].

2.1.4 Cache Architecture and Memory-Bound Parallelism

Modern shared-memory processors organize their caches in a hierarchy. Each core has private L1 and L2 caches—typically tens to hundreds of kilobytes—for low-latency access to recently used data. A shared last-level cache (LLC), often several megabytes, is accessible to all cores on the chip and serves as the final caching tier before main memory [1].

A workload is *compute-bound* when its execution time is dominated by arithmetic operations, leaving memory bandwidth underutilized. It is *memory-bandwidth-bound* (or simply *memory-bound*) when the bottleneck is the rate at which data can be fetched from DRAM rather than computational throughput. Pattern-matching algorithms that operate on large automata—whose transition tables exceed the LLC capacity—fall into this regime: each state transition triggers a cache miss that must be resolved from DRAM, making throughput a function of available memory bandwidth rather than CPU frequency or core count [10].

Parallelism can intensify memory pressure. When multiple threads simultaneously traverse different regions of a large, read-only automaton, the combined working set exceeds the LLC, raising the aggregate cache-miss rate for all threads. Two additional hazards arise when threads share writable data. *Cache coherence* protocols (such as MESI) ensure that modifications made by one core become visible to all others; they introduce inter-core invalidation traffic whenever a shared cache line is written [1]. *False sharing* occurs when two threads write to logically independent variables that happen to reside on the same cache line, causing coherence invalidations despite the absence of a true data dependency. False sharing can be avoided by aligning per-thread accumulators to cache-line boundaries.

Non-Uniform Memory Access (NUMA) architectures differentiate memory latency based on which socket owns the physical memory being accessed. Although the experimental platform used in this work—the Intel i5-1235U—employs a single-socket Uniform Memory Access (UMA) topology, NUMA is an important consideration in multi-socket server deployments where memory-bound workloads exhibit significant latency asymmetry [10].

2.1.5 Hybrid P/E-Core Architectures

Modern consumer and mobile processors have introduced a *hybrid* microarchitecture that integrates two distinct core types on a single die. Intel's Alder Lake platform—which includes the Intel Core i5-1235U used as the experimental platform in this work—pairs two high-performance cores (P-cores), each supporting Hyper-Threading (two logical hardware threads per core), with eight energy-efficient cores (E-cores), each providing a single hardware thread, for a total of twelve logical hardware threads [11].

P-cores feature wider out-of-order execution pipelines and higher operating frequencies, delivering greater per-thread throughput at higher power consumption. E-cores are physically smaller and optimized for throughput-per-watt rather than single-thread latency; they operate at lower frequencies. A memory-bound workload therefore achieves significantly higher per-thread throughput on a P-core than on an E-core.

Because the experimental platform of this work happens to be such a hybrid design, this heterogeneity is a property of the measurement environment rather than an object of study, but it does have a predictable effect on the observed scaling curves. At low thread counts, threads are preferentially dispatched to P-core hardware threads; speedup rises steeply because each additional thread contributes high-throughput capacity. Once all P-core hardware threads are occupied—four on the i5-1235U—additional threads are scheduled onto E-cores, which contribute lower per-thread throughput. The cumulative scaling curve therefore exhibits a *knee* near the P-core saturation point, beyond which the marginal speedup per additional thread declines. This is noted here only so that the scaling curves reported in the Results and Discussion chapter can be read in context; it also motivates the load-balancing strategy introduced in the proposed solution, whose purpose is to keep the heterogeneous cores from idling at the synchronization barrier.

2.2 PARALLEL PERFORMANCE EVALUATION METRICS

Performance evaluation is fundamental to understanding the effectiveness of a parallel program. The following metrics compare the execution time of the parallel program with its equivalent sequential counterpart [10]. The evaluation of parallel algorithms requires objective metrics that quantify both performance gains and resource utilization; accordingly, the metrics presented in this section provide the basis for evaluating the implementation proposed in this work and for interpreting the experimental results reported in Chapters 5 and 6.

2.2.1 Execution Time and Overhead

The performance of a program is measured by its execution time. For parallel programs, the following terms are defined:

- **Ts**: The execution time of a baseline sequential algorithm executed on a single processor unit.

- **Tp**: The execution time of the parallel program utilizing p processors.

Ideally, `Tp` is expected to be substantially less than `Ts`. However, parallel execution introduces *overhead*. This encompasses the time spent by processors on tasks that do not directly contribute to problem resolution, including communication, synchronization, load balance, and other computations inherent to the parallel implementation [10].

2.2.2 Speedup

Speedup (`S`) measures the performance gain achieved through parallel execution relative to sequential execution, as shown in Eq. 1. It is defined as the ratio of sequential execution time to parallel execution time:

$$S = \frac{T_s}{T_p} \tag{1}$$

An ideal, or linear, speedup would be `S = p`. Amdahl's Law [12] establishes a theoretical upper bound on the speedup achievable by parallelizing a program with a fixed problem size. If `s` represents the fraction of a program's sequential execution time that is inherently serial (non-parallelizable), and `f` is the parallelizable fraction (`s + f = 1`), the maximum speedup on `p` processors is given by Eq. 2.

$$S \leq \frac{1}{s + f/p} \tag{2}$$

In the limit, as the number of processors (`p`) tends to infinity, the maximum speedup is bounded by $\frac{1}{s}$. This implies that even a small sequential fraction (small `s`) can severely limit the total speedup, regardless of the number of processors available. Figure 1 illustrates the theoretical speedup according to Amdahl's Law compared to ideal speedup for a given sequential fraction.

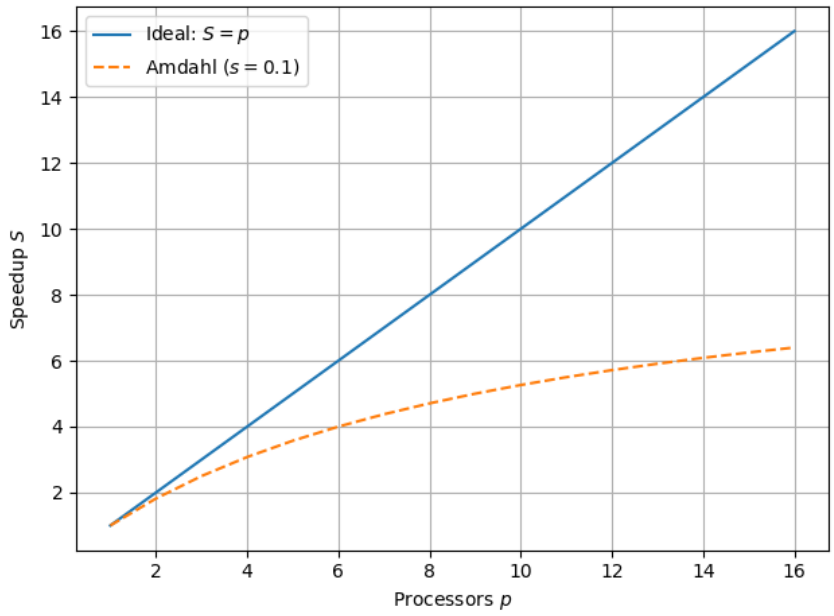


Figure 1: Ideal vs. Amdahl's theoretical speedup ($s = 0.1$).

A complementary perspective is offered by the Gustafson-Barsis Law [13] in Eq. 3, where α denotes the fraction of the *parallel* run time spent in non-parallelizable (serial) work (and is therefore a distinct quantity from Amdahl's s, which is the serial fraction of the *sequential* run). This law argues that, for problems whose size can be scaled with the available processors, an approximately linear speedup is attainable even in the presence of a serial fraction. Both Amdahl's and Gustafson's laws are important for understanding the limits and potential of parallelism in different scenarios [1].

$$S_{\text{scaled}} = \alpha + (1 - \alpha) \cdot p \tag{3}$$

2.2.3 Efficiency

Efficiency (E) measures the effectiveness of processor utilization in a parallel program. It is defined as the ratio of speedup to the number of processors p, as in Eq. 4:

$$E = \frac{S}{p} = \frac{T_s}{p \cdot T_p} \tag{4}$$

Efficiency values range between 0 and 1 (inclusive), often expressed as a percentage (0% to 100%). An efficiency of 1 (or 100%) signifies ideal, linear speedup, implying that all processors are contributing productively throughout the execution. In practice, efficiency typically diminishes as the processor count increases, primarily due to the escalating impact of parallel overhead [10].

2.2.4 Scalability

In parallel computing, scalability denotes a system's ability to sustain high perfor-

mance (often via efficiency) as both the number of processors and/or the problem size grow [10].

- **Strong Scalability**: Measures how execution time changes when the processor count p increases for a fixed total problem size, denoted W (for example, the total number of array elements to process). The goal is to keep efficiency nearly constant as p increases. Amdahl's Law directly applies in this scenario;
- **Weak Scalability**: Measures how execution time changes when both the processor count p and the total problem size W increase such that the work per processor (W/p) stays constant. The aim is to maintain efficiency as p and W grow proportionally. Gustafson's Law is most relevant here.

A parallel system is considered scalable if its efficiency remains above an acceptable threshold as both the number of processors p and the problem size W increase.

2.3 TRIES (PREFIX TREES)

This section explains the Trie data structure, detailing its defining properties, core operational mechanics, performance aspects, and its significance as a foundational element in advanced string processing algorithms, particularly the Aho-Corasick algorithm.

2.3.1 Definition and Basic Structure

A Trie—also known as a prefix tree—is a specialized tree-like data structure optimized for the efficient storage and retrieval of a dynamic set of strings. Edward Fredkin introduced the term "trie" in 1960, highlighting its primary application in information retrieval [14], [15]. Figure 2 illustrates an example of a trie storing a set of words.

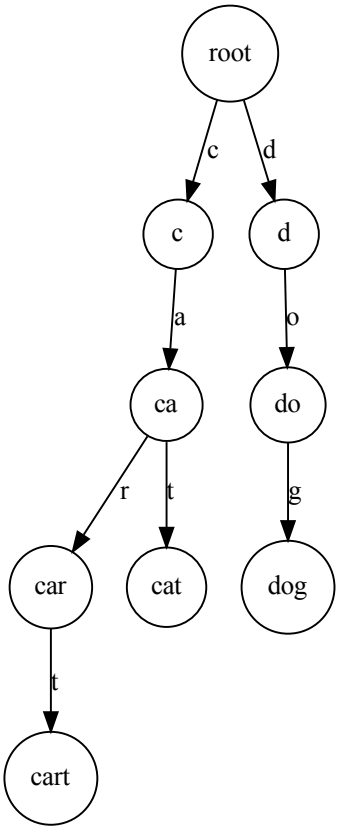


Figure 2: Example trie storing {“car”, “cat”, “cart”, “dog”}. Nodes marking the end of a complete word are shown with a double circle.

Tries store strings by organizing nodes so that each path from the root to a node represents a unique prefix. In a standard R-way trie, each node can branch to R children, where R denotes the size of the character alphabet. The existence of a parent-child edge highlights two key aspects of tries:

- The edge’s position (index) within the parent node’s array of R possible child links directly corresponds to a unique character in the alphabet. For instance, index 0 might correspond to ‘a’, index 1 to ‘b’, and so on, according to a predefined mapping.
- The character associated with this edge extends the prefix represented by the parent; this implies that at least one word (or key) in the trie contains this character sequence.

To signify that such a prefix also constitutes a complete key present in the trie, the node reached at the end of this path is typically marked by storing an associated non-

null value or a boolean flag. The characters themselves are implicitly encoded by the trie's structure rather than being explicitly stored within each node [16].

2.3.2 Core Operations

Tries support several primary operations, including search, insertion, and deletion [16]. These operations are explained below:

- **Search**: locating a key involves traversing the trie starting from the root, following the successive characters of the target key. Each character dictates which child link to follow. The search operation succeeds if a complete path corresponding to all characters of the key exists and the node reached at the end of this path is appropriately marked as representing a complete key. The search fails if, at any point, a required link is null (i.e., no path for the current character exists) or if the path is fully traversed but the final node is not marked as a complete key;
- **Insertion**: the insertion process begins similarly to a search, by traversing the trie according to the characters of the key to be added. If the path for the key does not extend to its full length (i.e., a null link is encountered), new nodes are created and linked to form the remainder of the path. The node corresponding to the final character of the inserted key is then marked to indicate that it is a complete key in the set;
- **Deletion**: deleting a key first involves locating the node corresponding to the key's final character and removing its end-of-word marker. If there are no child links (meaning it is not a prefix for any other key), it can be deleted. This deletion may proceed recursively: when a child node is removed, if its parent has no other children and the parent node is not marking a key, the parent node becomes redundant and can be deleted. This process continues up to the root as long as parent nodes become redundant.

2.3.3 Advantages and Performance Characteristics

Tries provide distinct advantages for operations on string collections, as detailed below.

- **Time Complexity**: a primary advantage is that the time required for search and insertion operations depends primarily on the length of the key (L), not on the total number (N) of keys stored. Specifically, these operations typically require examining a number of nodes proportional to L. Sedgewick and Wayne [16] state that the number of array accesses (node visits) is at most $L+1$. This $O(L)$ performance makes tries highly efficient for prefix-based searches (e.g., retrieving all keys starting with a given prefix) and ensures that search performance does not substantially degrade as the dataset size increases;

- **Prefix Sharing**: when stored strings share common prefixes, tries inherently exploit this by having those strings share the initial path from the root. This sharing can lead to significant space savings compared to storing each string independently, particularly if there is substantial prefix overlap among keys [14].

While prefix sharing can save space, the overall space complexity of R-way tries requires careful consideration. Each node in an R-way trie conceptually provides R potential links. If the alphabet size R is large (e.g., for Unicode) and the actual branching factor at most nodes is low (a sparse trie), allocating space for all R links per node can lead to considerable memory consumption due to numerous unused (null) links [15], [16].

In summary, the trie data structure is a cornerstone for numerous advanced string processing algorithms. Critically for this work, the Aho-Corasick algorithm, engineered for the efficient, simultaneous matching of multiple patterns within a text, employs a trie—constructed from the set of patterns (the dictionary)—as its initial structural framework [4]. A thorough grasp of the trie principles presented above is thus essential for developing and analyzing the Aho-Corasick algorithm.

2.4 PATTERN MATCHING

This section introduces the fundamental problem of pattern matching within strings, explores key algorithmic approaches to its solution, and highlights its significance across various computational domains. This foundational discussion provides the essential context for understanding advanced multi-pattern matching algorithms, such as the Aho-Corasick algorithm. In general information processing, pattern (or *string*) matching underpins the functionality of text editors and web search engines for keyword searches. The field of computational biology extensively utilizes pattern matching for analyzing genetic sequences [16]. Furthermore, in the field of information security, pattern matching is crucial: Network Intrusion Detection Systems (NIDS) utilize it to identify malicious signatures in network traffic. Tools like YARA, for instance, define rules based on textual or binary patterns to classify and identify malware [3].

2.4.1 The Pattern Matching Problem

The *string matching* problem consists of determining whether a string P (the *pattern*) of length M occurs within a string T (the *text*) of length N, where $M \leq N$. Formally, it seeks all positions pos such that the condition in Eq. 5 holds for every $i \in \{0, 1, \ldots, M-1\}$, and each valid pos corresponds to an occurrence of the pattern in the text:

$$P[i] = T[pos + i], \quad \text{with} \quad pos + i < N \tag{5}$$

Beyond simply finding the first match, the problem often extends to locating all occurrences, enumerating them, or extracting surrounding contextual information for each

match [16].

Algorithms that address the string matching problem are generally categorized based on the number of patterns they process and the exactness of the match. This thesis focuses on *Exact Matching*, where the pattern must correspond precisely to a segment of the text. A key distinction for this study lies between algorithms designed for *Single-Pattern Matching* and those optimized for *Multi-Pattern Matching*, where the objective is to identify all occurrences of any pattern from a predetermined set of patterns within a given text [4], [17].

2.4.2 The Naive Algorithm

The naive or brute-force algorithm represents the most straightforward method for exact pattern matching. Its operation involves iteratively aligning the pattern P with the text T at every possible starting position i (from 0 to $N - M$). For each alignment, P is compared character by character, from left to right, against the corresponding substring. If all characters match, an occurrence is reported [17].

Following this comparison (whether a match or a mismatch), the algorithm shifts P one position to the right relative to T and proceeds to the next alignment. Despite its simplicity, Knuth, Morris, and Pratt (1977) state that this method can be highly inefficient [18]. Its worst-case time complexity is $O(M \cdot N)$ character comparisons. This occurs, for example, when searching for a pattern like $P = a^n b$ within a text $T = a^{2n} b$, where many partial matches occur. To address these limitations, more advanced pattern matching algorithms include a preprocessing phase for the pattern P. This phase extracts structural information about P that is then utilized during the search, enabling larger shifts of the pattern relative to the text and thus reducing the total number of character comparisons.

2.4.3 Knuth-Morris-Pratt (KMP) Algorithm

The KMP algorithm [18] efficiently resolves the pattern matching problem with an optimal worst-case time complexity of $O(N + M)$. Unlike the naïve approach, KMP avoids re-checking characters that have already been matched. To achieve this, KMP first preprocesses the pattern P by constructing a special array known as the *failure array* (or *prefix array*), typically denoted by $p[1..M]$. This array indicates the length of the longest prefix that is also a suffix for each prefix of P. With this information, when a mismatch occurs during the search, the algorithm uses the failure array to decide which pattern index to resume comparison from, so the text pointer never backtracks, bypassing unnecessary comparisons and preventing redundant checks.

2.4.4 Boyer-Moore (BM) Algorithm

The Boyer-Moore (BM) algorithm [19] is widely recognized for its excellent practical performance, often outperforming linear-time methods in typical cases. Unlike the KMP algorithm, Boyer-Moore typically compares characters starting from the end of the pattern and moves backward. Its efficiency primarily stems from two rules (or heuristics):

- **Bad Character Rule**: When a mismatch occurs, this rule uses the character from the text that caused the mismatch to determine how far the pattern can safely shift. The algorithm shifts the pattern to align this mismatched text character with its last occurrence in the pattern or moves completely past the mismatched character if it does not appear in the pattern;
- **Good Suffix Rule**: When a mismatch happens after matching one or more characters from the end of the pattern, this rule uses the matched suffix to decide how far to shift the pattern. It searches for another occurrence of the matched suffix within the pattern, ensuring that no previously matched characters are unnecessarily compared again.

These two rules combined enable the Boyer-Moore algorithm to skip larger portions of the text, significantly reducing the total number of comparisons.

2.4.5 The Need for Specialized Multi-Pattern Algorithms

When the task involves searching for multiple patterns simultaneously from a set $K = \{P_1, P_2, \ldots, P_k\}$ within a text T, simply applying a single-pattern matching algorithm for each P_i independently proves highly inefficient. This "iterative" approach requires scanning the text multiple times, resulting in a total time complexity that can be approximately $O(k \cdot (N + M_{\text{avg}}))$ or worse, where M_{avg} is the average pattern length [4].

To efficiently handle applications that require simultaneous searches for a large number of patterns, specialized algorithms are essential. These methods usually preprocess the entire set of patterns to create a unified data structure, which enables a single, efficient pass over the text to identify all occurrences of any pattern within the set [17].

The theoretical framework of *finite automata* provides a robust and elegant solution to this problem. By constructing a single *finite automaton* (FA) that can recognize all patterns in a given set, the text can be processed in a single continuous scan. Each character read from the text triggers a *state transition* within the FA. When the FA reaches an *accepting state*—a state explicitly designed to indicate the successful recognition of one or more patterns—the algorithm reports an occurrence. Algorithms like the Aho-Corasick algorithm exemplify this approach; they automate the construction of such a *pattern-matching machine*, typically based on a trie structure augmented with specialized *failure transitions* [4]. This automaton-based paradigm forms the critical foundation for efficiently solving multi-pattern matching problems. The Aho-Corasick algorithm,

which instantiates this paradigm and is the focus of this work, is examined in detail in the next section.

2.5 THE AHO-CORASICK ALGORITHM

This section provides a comprehensive examination of the Aho-Corasick algorithm, a highly efficient string-matching method designed to locate all occurrences of multiple patterns (keywords) within a given text. It combines *automata theory* with *trie* data structures to efficiently solve the *multi-pattern search* problem. Its search phase runs in $O(n + z)$ time, where n is the text length and z is the number of matches [4].

2.5.1 Context

The task of concurrently identifying all instances of a large collection of patterns within a text poses a significant computational challenge. A naïve approach—running a *single-pattern matching* algorithm such as KMP or BM for each of the k patterns—requires repeated scans of the text and has a worst-case time complexity of $O(k \cdot n)$, where k is the number of patterns and n is the text length, rendering it impractical for large pattern sets or long texts [17].

The Aho-Corasick algorithm overcomes this by building one *finite automaton* that encodes all patterns simultaneously. This automaton processes the input text in a single pass to detect all matches. The algorithm effectively combines the *state-transition* efficiency of finite automata with the prefix-sharing advantages of tries, following *failure links* on mismatches and resuming matching without rescanning characters—an extension of KMP's *failure-function* idea to a trie of patterns [17].

2.5.2 Algorithm Components and Construction

Aho-Corasick has two main phases: (i) construction, which builds the automaton, and (ii) search, which scans the text. The automaton has three key tables, built in the following order:

- **Keyword Trie (goto)**: a *keyword trie* is first constructed from the pattern set $K = \{P_1, P_2, \ldots, P_k\}$. Each node represents a state, and each edge corresponds to a character transition. A path from the root to any state s spells a prefix of one or more keywords. Formally, the *goto function* goto(s, a) returns the next state s' when an edge labelled a exists; otherwise the transition fails. Undefined transitions from the root are assigned back to the root, i.e., goto$(0, a) = 0$, creating implicit self-loops for every alphabet symbol;

- **Failure Function (f)**: for any state s (except the root), f(s) is the state s' whose path label is the longest proper suffix of the path label of s that is also a prefix of some keyword. On a mismatch, the automaton follows f(s) and resumes matching

without consuming the current character. The *failure function* is computed level-by-level (breadth-first) after the goto table is complete;

- **The Output Function**: the *output function*, output(s), identifies all keywords that end at state s. For a given state s, output(s) is the set of all patterns P_i in K such that the string corresponding to the path to s is P_i. To ensure all matches are reported, including those that are suffixes of other patterns, output(s) is augmented during the failure function computation: if f$(s) = s'$, then output$(s) \leftarrow$ output$(s) \cup$ output(f(s)). This incremental union lets each state report every pattern that ends either at that state or at any suffix state reached through its failure chain.

The complete automaton, combining the goto transitions, failure links, and output match sets, is illustrated in Figure 3 for the pattern set $K = \{$”he”, ”she”, ”hers”$\}$.

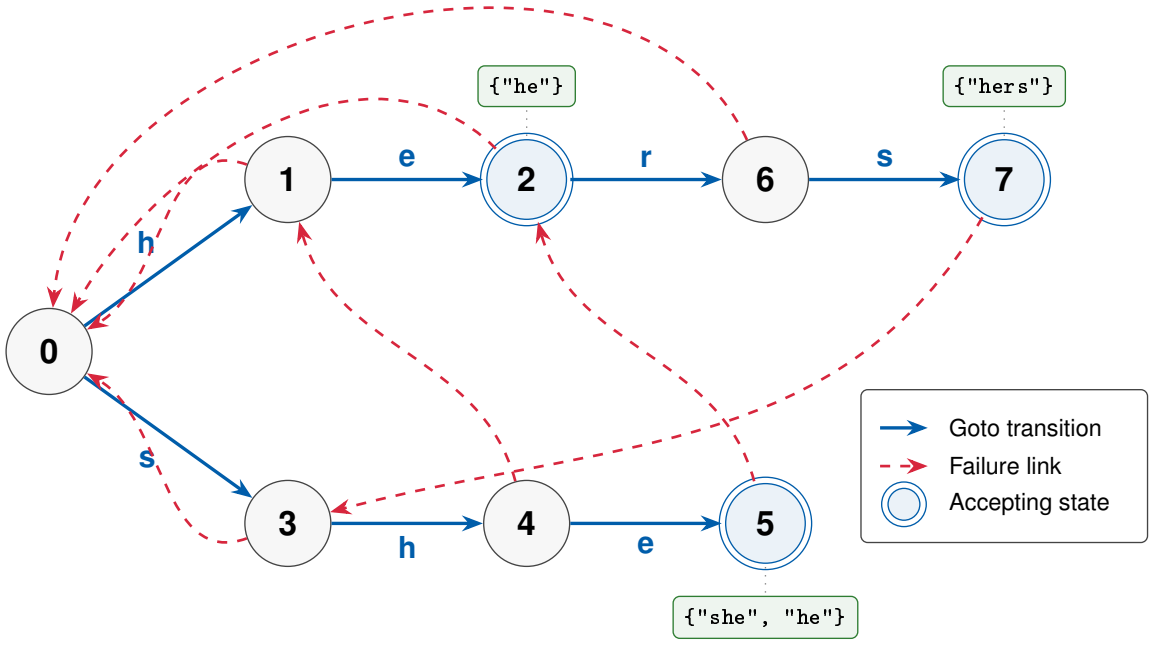


Figure 3: Complete Aho–Corasick automaton for the pattern set $K = \{$”he”, ”she”, ”hers”$\}$, showing solid blue goto transitions, dashed red failure links, double-circled accepting states, and their corresponding output match sets.

To make the interaction of the three tables concrete, consider scanning the text `ushers` with the automaton of Figure 3. The leading `u` has no outgoing edge from the

root, so the implicit self-loop keeps the machine at state 0. The substring `she` then drives three successive goto transitions to the accepting state for `she`; because the failure link of that state points to the state for `he`—the longest proper suffix of `she` that is also a keyword prefix—its augmented output set reports both `she` and `he` at that position. The next character `r` has no goto edge from the `she` state, so the automaton follows the failure link to the `he` state and continues along `her` and then `hers`, emitting `hers` when the final `s` is consumed. A single left-to-right pass thus reports every occurrence—`he`, `she`, and `hers`—without ever rescanning a character.

2.5.3 Matching Phase

Once the Aho–Corasick automaton has been built, it scans the text $T = a_1 a_2 \dots a_n$ in one left-to-right pass. At each position i, the machine attempts to follow the transition labelled a_i from its *current_state* (cs); if that edge exists, it moves along it, otherwise it follows *failure links* (f) until either a matching transition is found or the root is reached. Every time a successful transition is made, the automaton inspects the output set of the new state and emits all patterns ending at position i.

Figure 4 illustrates this control flow in detail, which: starts at the root with $i = 1$, attempt each goto(*current_state*, a_i), follow failure links on a “no” decision, report matches via output(*current_state*) on a “yes,” advance i, and terminate when $i > n$.

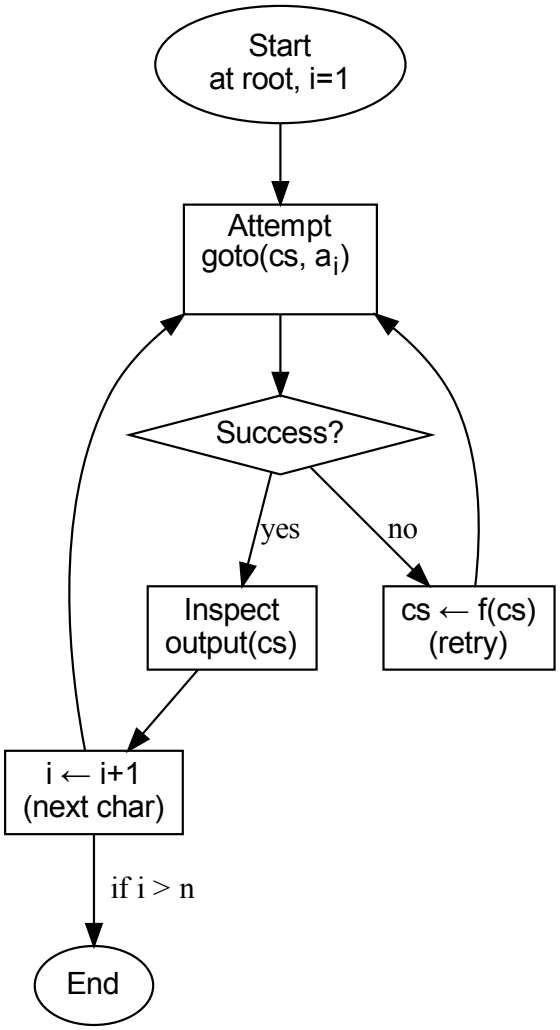


Figure 4: Flowchart of the Aho–Corasick matching phase.

Each input character advances the automaton by at most one goto transition, which increases the current depth by one, whereas every failure-link traversal strictly decreases that depth. Since the depth is non-negative, the total number of failure transitions over the scan cannot exceed the number of goto transitions, which bounds the total number of transitions (goto plus failure) by $< 2n$ [4].

2.5.4 Time and Space Complexity

The Aho-Corasick algorithm exhibits optimal *linear time complexity* for multi-pattern matching [4], [17]. The complexities for the construction and search phases and storing the automaton are as follows:

- **Construction phase**: building the goto, failure, and output functions takes $O(L)$

time, where L is the total length of all keywords. If a dense array is used for the root, an additional $O(|\Sigma|)$ initialization cost is incurred, giving $O(L + |\Sigma|)$;

- **Search phase**: Scanning a text of length n performs fewer than $2n$ state transitions. Reporting matches costs time proportional to the total number of occurrences z, so the total search time is $O(n + z)$;
- **Space complexity**: Storing the automaton requires $O(L + |\Sigma|)$ space: $O(L)$ for trie edges, $O(L)$ pointers for failure links, and additional space for output lists (proportional to the number of stored outputs).

2.5.5 NFA versus DFA Representations

The goto-plus-failure automaton described above is commonly distinguished, in the network-intrusion-detection literature, as the *nondeterministic finite automaton* (NFA) configuration of Aho–Corasick, in contrast to the fully precomputed *deterministic finite automaton* (DFA) discussed below [20]. This NFA label is conventional rather than formal—the goto-plus-failure machine in fact behaves deterministically—but it captures the operational difference: the goto function is undefined on many (state, character) pairs, and the search algorithm resolves those gaps by traversing failure links at runtime, so a single character may require several memory accesses in the worst case (the cost is constant only in the amortized sense established above). A widely adopted optimization converts this NFA into a *deterministic finite automaton* (DFA) by precomputing the failure-link chains during construction: for every state s and character a, the full deterministic transition $\delta(s, a)$ is stored in a flat two-dimensional table of size $|Q| \times |\Sigma|$, where $|Q|$ is the number of states and $|\Sigma|$ is the alphabet size [4], [17]. With this representation every character in the input triggers exactly one array lookup; no runtime failure-link traversal occurs, which reduces per-character latency and simplifies the inner search loop.

The cost of the DFA representation is memory: for the standard 256-symbol byte alphabet the flat transition table occupies $4 \cdot |Q| \cdot 256$ bytes. This footprint grows linearly with the pattern-set size and can easily exceed the processor's last-level cache (LLC). When the table no longer fits in cache, each state transition becomes a potential cache miss, shifting the workload from compute-bound to *memory-bandwidth-bound*. The implementation in this work uses the DFA (flat transition table) representation exclusively; the goto function, failure function, and output function described in the preceding subsection are intermediate construction artefacts that are not retained at search time.

2.5.6 Applications and Relevance

The Aho-Corasick algorithm is widely used in applications that require high-performance *multi-pattern matching*, where its single-pass processing of the input against the en-

tire pattern set is essential. Representative use cases include large-scale text analysis, motif searching in DNA or protein sequences [5], spam filtering, Network Intrusion Detection Systems (NIDS) that scan traffic against thousands of signatures, and anti-malware tooling—YARA, for example, employs Aho-Corasick to match the literal string components extracted from its rules [3].

While Aho-Corasick excels at exact literal matching, systems like NIDS often layer additional capabilities—such as *regular-expression matching*—on top of Aho-Corasick engines to support more expressive signatures, which affects overall performance characteristics. The algorithm therefore remains a theoretically elegant and practically impactful example of *finite-automata theory* applied to string processing [17], [3].


3

3 SYSTEMATIC REVIEW

This section outlines the methodology employed for the Systematic Literature Review (SLR) conducted to identify, evaluate, and synthesize pertinent studies concerning the parallelization and optimization of the Aho-Corasick algorithm. The review focuses on implementations targeting multi-core CPUs. The objective of this SLR is to support the development of the present research and to answer the formulated research questions.

3.1 RESEARCH QUESTIONS

The following Research Questions (RQs) were formulated to guide the literature search, selection, and analysis process:

1. **RQ1:** What parallelization techniques and associated challenges are documented in the literature for implementing the Aho-Corasick algorithm utilizing threads on multi-core CPUs?

2. **RQ2:** What specific optimizations for the Aho-Corasick algorithm are employed in application scenarios involving extensive pattern sets, particularly within domains of information security?

These research questions were formulated based on the objectives of this work and aim to identify both parallelization strategies and optimization techniques applicable to the proposed implementation.

3.2 SEARCH STRATEGY AND STUDY SELECTION PROTOCOL

The search and selection protocol for identifying primary studies encompassed the definition of data sources, construction of search queries, and establishment of inclusion and exclusion criteria.

3.2.1 Data Sources

The following scientific databases were selected due to their scope and relevance in the area of Computer Science and Software Engineering:

- Web of Science (WoS)
- Scopus

- ACM Digital Library (ACM DL)
- IEEE Xplore

3.2.2 Search Queries

Two primary search strings were constructed to address the aspects of parallelization (RQ1) and optimization/application (RQ2) pertinent to the Aho-Corasick algorithm. The literature search was executed in May 2025.

String 1 (Focus on Parallelization):

```
"Aho-Corasick" AND (parallel* OR multithread* OR multi-thread* OR thread* OR concurren* OR hpc OR high-perform* OR high perform*) AND ("multi-core" OR multicore OR "many-core" OR CPU) NOT "distributed systems"
```

String 2 (Focus on Optimization and Applications):

```
"Aho-Corasick" AND (optimi* OR accelerat* OR perform* OR evaluat* OR improve*) AND ("application" OR "implementation" OR "use case")
```

3.2.3 Inclusion (IC) and Exclusion (EC) Criteria

The subsequent criteria were established for the selection of primary studies:

Inclusion Criteria (IC):

- **IC1 (Primary Focus):** The study addresses the Aho-Corasick (AC) algorithm or presents parallelization/optimization techniques demonstrably applicable to AC.
- **IC2 (Parallelism Aspect):** The study investigates, proposes, implements, analyzes, or explicitly discusses the parallelization or concurrent execution of the Aho-Corasick algorithm.
- **IC3 (Target Platform):** The study reports empirical results or analyses about multi-core CPU platforms.
- **IC4 (RQ Relevance):** The study provides information relevant to addressing at least one RQ.
- **IC5 (Language):** Published in English or Portuguese.
- **IC6 (Accessibility):** Full-text availability of the study.
- **IC7 (Publication Window):** Published between January 2015 and May 2025, inclusive.

Exclusion Criteria (EC):

- **EC1 (Irrelevant Focus):** Studies with only superficial mentions of the Aho-Corasick algorithm or pattern matching, lacking a primary focus on its parallelization or optimization.
- **EC2 (Non-CPU Platform Exclusivity):** Studies exclusively focused on non-CPU platforms (e.g., GPU, FPGA) without analysis or insights transferable to multi-core CPU environments.
- **EC3 (Insufficient Detail):** Studies lacking sufficient detail regarding methodology, implementation, or reported results.
- **EC4 (Publication Type):** Opinion pieces, editorials, and non-peer-reviewed gray literature.
- **EC5 (Language Barrier):** Publications in languages other than English or Portuguese.
- **EC6 (Duplication):** Duplicate publications (the most comprehensive or recent version was retained).
- **EC7 (Inaccessibility):** Full-text unobtainable.

3.2.4 Selection Process and Initial Search Outcomes

The study selection was a multi-stage process. Initially, the constructed search strings were executed against the selected digital libraries. The retrieved results were subsequently refined using filters provided by each database (e.g., publication period, document type, language, subject area), as detailed in Table 1:

Table 1: Search results and filters applied per database.

Database	**String 1** (Found / Kept)	**String 2** (Found / Kept)	**Filters Applied**
ACM Digital Library	122 / 29	274 / 66	2015-2025; Research Article; Publisher: ACM; Title excl.: GPU; Lang: English.
Web of Science	30 / 6	66 / 20	2015-2025; WoS Cats: CS (excl. AI); Title excl.: GPU; Lang: English.
Scopus	34 / 6	145 / 49	2015-2025; Lang: English; Subj: CS, Eng; Types: Article, Conf. Paper; Title excl.: GPU.
IEEE Xplore	25 / 2	60 / 18	2015-2025; Title excl.: GPU; Lang: English.

The `Title excl.: GPU` filter listed in Table 1 acted only as a coarse identification-stage noise reducer, not as the formal platform criterion: the platform decision is governed by EC2, under which studies on non-CPU platforms (GPU, FPGA, ASIC, SIMD) were retained whenever they offered techniques or findings transferable to shared-memory multi-core CPUs, as argued individually in each review below.

Following the initial database search and filtering stage, the resultant sets of candidate articles were aggregated. The 196 records retained across both search strings and all four databases (Table 1) were merged; after removing duplicate entries and screening the titles and abstracts of the remainder against the inclusion and exclusion criteria, 24 unique studies were pre-selected. From these 24, the 10 most directly pertinent to the research questions were judged most relevant and were summarized as the primary studies for the detailed review that follows.

3.3 REVIEW OF SELECTED STUDIES

The selection process described above yielded the corpus of primary studies analyzed in this review. The following subsections examine in detail the ten selected primary studies, spanning shared-memory multi-core parallelizations of Aho–Corasick (RQ1) and optimizations of the algorithm in security-oriented matching workloads (RQ2); each review summarizes the problem addressed, the proposed technique, the experimental evidence, and its relation to the present design.

3.3.1 A Parallel Algorithm of String Matching Based on Message Passing Interface for Multicore Processors

Qu et al. [20] address the limitations of classical multiple-pattern string matching algorithms, such as the Aho-Corasick algorithm, which were originally designed for single-core systems and often become bottlenecks in modern network intrusion detection systems under heavy traffic. The authors highlight the need to use the parallel computation power of modern multicore processors to maintain high performance in deep packet inspection.

To solve this performance bottleneck, the researchers propose a parallel Aho-Corasick string matching algorithm that runs on a multicore platform but coordinates its workers through the Message Passing Interface (MPI), a distributed-memory programming model applied here within a single shared-memory node. The core methodology relies on data parallelism: a master thread constructs the automaton sequentially and partitions the input text into contiguous blocks. To prevent missed cross-boundary matches, these blocks overlap by the length of the longest pattern minus one character. The data chunks are then distributed to multiple slave threads via MPI communication, allowing each worker to independently traverse the read-only automaton.

The proposed solution was evaluated on an 8-core multicore system using Snort IDS

rulesets of 10,000 and 30,000 patterns. The results demonstrate that the Deterministic Finite Automaton (DFA) variant achieves a peak average throughput of 10.5 Gbps, outperforming the Nondeterministic Finite Automaton (NFA) approach in speed and stability, albeit with higher memory consumption. Furthermore, the evaluation shows that throughput scales monotonically with the addition of CPU cores and thread-level parallelism.

Ultimately, the paper demonstrates that a master-slave data partitioning strategy with overlapping boundaries is highly effective for parallelizing the Aho-Corasick algorithm on multicore processors. However, the reliance on MPI for a single shared-memory node introduces serialized communication overhead, which highlights the advantages of using native shared-memory threading models, such as POSIX Threads, for lock-free coordination.

3.3.2 A Flexible Pattern-Matching Algorithm for Network Intrusion Detection Systems Using Multi-Core Processors

Lee and Yang [2] tackle the throughput bottleneck imposed by Aho-Corasick-based pattern matching in signature-based Network Intrusion Detection Systems (NIDS), where the matching phase can account for up to 70% of total execution time. The authors observe that, although the canonical Aho-Corasick Deterministic Finite Automaton (AC-DFA) offers linear-time scanning, its state-transition table causes heavy memory traffic and frequent last-level cache misses as the pattern set grows. The prior Head-Body Matching (HBM) algorithm mitigates this by partitioning the AC-DFA into a fast head (DFA) and a SIMD-accelerated body (NFA), but its rigid depth-based partitioning criterion typically overshoots or undershoots the cache budget, leading to suboptimal throughput.

To overcome this rigidity, the researchers propose the Flexible Head-Body Matching (FHBM) algorithm. Instead of partitioning the automaton at a fixed depth, FHBM employs a greedy procedure that fills the head with up to a configurable maximum number of states (`HSIZE`), prioritizing parent nodes with the largest number of children and preserving the invariant that all siblings remain on the same side of the partition. The head is represented as a full 256-entry transition table for $O(1)$ lookup, while the body is compacted into an NFA traversed via 128-bit SIMD comparisons. Parallel execution is realized at the thread level: four worker threads, one per physical core, concurrently traverse the read-only Head-Body Finite Automaton (HBFA) on the input stream, exploiting the immutability of the automaton after construction to avoid synchronization in the search phase.

The evaluation was conducted on a quad-core Intel Core i7-3770 (3.4 GHz, 8 MB L3) running 64-bit Ubuntu 14.04, using pattern dictionaries extracted from the Snort ruleset (8,673 patterns) and the ClamAV signature database (5,248 and 2,899 patterns for the

type-1 and type-3 sets, respectively). Synthetic input streams were generated by embedding pattern prefixes into a clean corpus – the King James Bible (HTML) for Snort and the concatenation of `/usr/bin` binaries for ClamAV – with controlled match ratios of 1%, 8% and 32%. The reported throughput in MB/s, averaged over 100 runs, shows that FHBM consistently outperforms HBM where the partition matters: gains of approximately +58% on Snort (536 versus 339 MB/s) and +46% on ClamAV type-1 (678 versus 465 MB/s) at a 1% match ratio, and +55% on ClamAV type-3 (541 versus 349 MB/s) at a 32% match ratio. The absolute peaks, where FHBM and HBM converge, are instead governed by the cache: Snort plateaus near 787 MB/s once the head reaches $\approx 17{,}000$ states—the point at which its ≈ 8.5 MB footprint saturates the 8 MB L3 and additional states stop helping—while ClamAV type-3, whose head is far smaller ($\approx 5{,}000$ states), exceeds $1{,}300$ MB/s at a 1% match ratio.

Overall, the paper demonstrates that a flexible, state-count-based partitioning of the AC-DFA, combined with SIMD body evaluation and multi-core data parallelism, achieves good throughput on commodity hardware. It also identifies cache capacity as the dominant performance ceiling: once the active state memory exceeds the L3 budget, additional states no longer improve throughput, a finding that reinforces the cache-locality concerns inherent in any shared-memory parallel implementation of Aho-Corasick. Two architectural choices, however, contrast with the present POSIX Threads approach: the authors do not explicitly describe how the input stream is partitioned across the four worker threads nor how cross-boundary matches are handled, and their speedup is tightly coupled to platform-specific SIMD intrinsics rather than to portable data-parallel chunking with overlap. This positions FHBM as a complementary, automaton-restructuring strategy whose throughput gains stem from microarchitectural exploitation rather than from explicit input partitioning, providing a useful comparative reference point for the present Pthreads-based design.

3.3.3 An Optimized Parallel Failure-less Aho-Corasick Algorithm for DNA Sequence Matching

Thambawita et al. [5] investigate the throughput limitations of the canonical Aho-Corasick (AC) algorithm in bioinformatics workloads, where the dictionary may contain thousands of DNA patterns and the input may reach hundreds of megabytes per scan. The authors observe that, although the Parallel Failure-less Aho-Corasick (PFAC) library of Lin et al. already exposes massive thread-level parallelism by launching one thread per input character, it remains a domain-agnostic implementation whose 256-entry transition table and dispersed memory layout do not exploit two structural properties of DNA matching: the alphabet contains only four symbols ($\{A, T, C, G\}$) and the matching kernel has strong spatial and temporal locality. As a result, the baseline PFAC implementation underuses memory bandwidth and has poor cache behaviour on the

GPU memory hierarchy.

To overcome these limitations, the authors propose an application-specific optimization of PFAC on a NVIDIA Fermi GPGPU with three coordinated changes. First, the AC transition table is reduced from a generic $256 \times N$ matrix to a compact $4 \times N$ table aligned with the DNA alphabet, drastically shrinking the working set. Second, the transition table is mapped onto texture memory to exploit its two-dimensional spatial caching, while the input text is staged into shared memory to exploit temporal locality across the threads of each block. Third, the `nextState` and `matchedPatternId` arrays are merged into a single contiguous one-dimensional array so that consecutive accesses benefit from cache-line reuse. The underlying parallel decomposition is preserved from PFAC: every input character is assigned its own thread, which independently traverses the failure-less automaton until a mismatch terminates its scan, and matches are recorded in thread-local result slots that require no inter-thread synchronization during the search phase.

The evaluation is conducted on a NVIDIA Tesla C2075 (Fermi architecture), using five synthetic DNA dictionaries (ranging from $1{,}000$ to $5{,}000$ patterns) and five synthetic DNA input datasets sized between 76 MB and 380.2 MB. The reported metric is wall-clock execution time, measured against the unmodified PFAC library; absolute throughput in MB/s and any sequential CPU baseline are absent. Across all configurations, the optimized implementation consistently outperforms the original PFAC, with the largest datasets exhibiting approximately a $3\times$ speedup over the baseline library. The authors further observe that memory-layout optimizations – in particular the merged one-dimensional array and the use of texture memory – have a substantially larger impact on performance than tuning L1 cache sizes or per-block thread configurations.

This study reinforces two principles that are directly transferable to the present Pthreads-based design even though the target hardware differs: first, that the read-only nature of the AC automaton after construction enables fully lock-free, thread-local matching, irrespective of whether parallelism is realized on a GPU or on a multi-core CPU; second, that the placement of the automaton and the input within the cache hierarchy often matters more than raw thread-count scaling. Two limitations, however, restrict the direct applicability of these findings to the current work. The PFAC decomposition launches one thread per input byte, which is well suited to GPU SIMT execution but mismatched with the much coarser granularity of multi-core CPUs, where the Pthreads model favours chunk-based data partitioning with overlap regions of `max_pattern_len`-1 bytes. Moreover, the alphabet-specific reduction to a $4 \times N$ transition table relies on the small DNA alphabet and cannot be reused for the full 256-symbol byte alphabet of IDS/YARA workloads targeted by the present implementation. The Thambawita et al. study thus serves as a complementary point of comparison, framing PFAC-style fine-grained parallelism as the GPU counterpart of the coarse-grained, chunked Pthreads

strategy adopted in this thesis.

3.3.4 The Diversification and Enhancement of an IDS Scheme for the Cybersecurity Needs of Modern Supply Chains

Deyannis et al. [21] examine the performance and energy cost of software Aho-Corasick matching when an industrial supply chain node must concurrently execute its primary business workload and inspect real-time network traffic. The authors observe that the canonical Snort IDS, although well-tuned, saturates the CPUs of low-power supply chain assets and therefore introduces a deployment gap: edge nodes either drop packets under load or shift inspection costs upstream, which conflicts with the distributed perimeter defence recommended by recent supply chain cybersecurity frameworks. The bottleneck is clearly in the multi-pattern matching kernel, which Snort implements with Aho-Corasick.

To restore line-rate operation without altering the IDS configuration model, the authors offload the AC engine to the FPGA fabric of a low-cost PYNQ Z1 board (Xilinx Zynq-7000 MPSoC). The fully expanded, failure-free DFA is serialized as a flat two-dimensional integer array indexed by `dfa[state * 256 + char]`, eliminating pointer-chasing and turning every transition into a single arithmetic lookup – a layout structurally identical to the read-only `goto_tbl` adopted by the present Pthreads implementation. The transition table is then fed into a four-stage hardware pipeline implemented in Vivado HLS and clocked at 410 MHz, while the official Snort software stack continues to run on the co-located ARM Cortex-A9 cores; only the AC matching kernel is replaced. Because the design consumes less than 10% of the available LUTs, the authors note that two or more copies of the IP module could be instantiated to expose additional spatial parallelism, projecting theoretical $2.8\times$ and $5.6\times$ accelerations that, however, are not empirically validated.

The evaluation employs four publicly available IIoT traffic datasets – the Malware Traffic Analysis blog capture, DARPA-99, the UNSW ToN_IoT dataset and the NCC Group honeypot trace – replayed against the same Snort ruleset, comparing the FPGA-accelerated build against a fully optimized software Snort baseline running on the identical PYNQ board. The reconfigurable design delivers $1.2\times$ to $1.4\times$ acceleration in pure execution time (e.g., 79.4 s versus 108.9 s on the DARPA-99 trace) and up to $1.4\times$ higher packet-per-second rates on traces dominated by large packets, while underperforming the software baseline by approximately 10% on small-packet traces because of interrupt and DMA overhead. The total board power draw of ≈ 13.8 W is much lower than the ≈ 118 W of a high-end single-socket server; combined with the up-to-$1.4\times$ acceleration, the authors report an energy consumption up to almost $12\times$ lower than that server. Aggregate throughput in Gbps and the exact size of the loaded ruleset are not disclosed.

In sum, the paper confirms that the matching phase of Aho-Corasick is the dominant cost of signature-based IDS in resource-constrained settings and demonstrates that a hardware-pipelined DFA can recover throughput at a fraction of the energy budget. Three observations transfer directly to the present Pthreads-based design: the use of a fully expanded failure-free DFA stored as a flat $N \times 256$ integer array is equally appropriate for shared-memory CPU traversal, since it preserves $O(1)$ per-character transitions while permitting concurrent read-only access by multiple worker threads; the matching phase rather than the automaton construction dominates the cost, justifying the architectural decision to parallelize only the search loop; and packet-size or input-size sensitivity is a workload effect that any chunk-based parallel implementation should evaluate, since small inputs amortize less of the per-chunk start-up cost. The principal divergence is architectural: while the paper realizes parallelism through hardware pipelining and replicated FPGA IP cores, the present Pthreads design pursues coarse-grained data parallelism through chunked partitioning of the input with overlap regions of $\texttt{max\_pattern\_len} - 1$ bytes, exploiting commodity multi-core CPUs and avoiding the reconfigurable-hardware deployment cost.

3.3.5 Fast String Searching on PISA

Jepsen et al. [22] target the bandwidth and per-byte compute bottlenecks that arise in disaggregated data centres, where storage nodes carry weak CPUs and every byte of incoming data must traverse PCI-Express to reach a general-purpose processor. The authors observe that pre-filtering large volumes of stored or in-flight data with multi-pattern string matchers is essential to keep downstream analytics manageable, but that an x86 host inspecting every byte does not scale to disk-array or line-rate workloads. Aho-Corasick is the standard algorithm for this pre-filter, and the paper poses the question of whether its deterministic finite automaton can be executed entirely within the data plane of a programmable network ASIC, exploiting native pipeline parallelism rather than thread-level parallelism on a CPU.

To answer this question, the authors introduce PPS, a P4-based compiler and runtime that maps an Aho-Corasick-derived DFA to the PISA architecture of the Barefoot Tofino switch. Two transformations are applied to the canonical automaton. First, a k-stride DFA is created to process k bytes at a time, so that each pipeline pass consumes several characters at the cost of a multiplicatively larger transition table; this trades on-chip SRAM for higher per-stage throughput. Second, the resulting DFA is optionally partitioned into up to three independent sub-automata that are placed in different pipeline stages and traversed in parallel against the same packet, allowing the aggregate pattern set to exceed the memory of any single stage. To enforce correctness when packets are processed independently, the system mandates that adjacent input chunks overlap by the length of the longest pattern – a constraint conceptually

identical to the `max_pattern_len` $- 1$ overlap adopted by the present Pthreads implementation, although applied here at the network MTU granularity. An optional CRC16 hash representation of the transition table is offered as an accuracy-for-memory trade-off; this approximation is incompatible with exact-match requirements and is therefore restricted to use cases tolerant of false positives.

The evaluation contrasts PPS running on a Tofino switch against two CPU baselines – Spark SQL on a four-node dual-socket Intel Xeon E5-2603 cluster and Unix `grep` on a 12 GB RAM-disk log – using Snort community-rule `content` patterns of up to 32 characters and three corpora: the 4.7 GB Podesta e-mail archive, a 212 GB Twitter streaming capture, and the synthetic log file. End-to-end execution times measured at 128 patterns yield a $> 20\times$ speedup over Spark SQL on the e-mail workload and a $6.5\times$ speedup on the tweet workload (5.43 min versus 35.4 min), while throughput at 100 Snort patterns with stride 4 is calculated – not measured – at 3.8 Tbps for a 64-port Tofino. The authors are explicit that the Tbps figure is a theoretical upper bound derived from the deterministic line rate of the compiled P4 program; CPU baselines are measured end-to-end, and external GPU/FPGA references (e.g., 150 Gbps on GPU, 45 Gbps for DFC on x86) are quoted rather than reproduced.

The paper shows that Aho-Corasick is an algorithm that scales from a single CPU core all the way to multi-Tbps switching silicon, and it makes two design decisions that map directly onto the present Pthreads-based work. The mandatory overlap of *longest-pattern* bytes between adjacent input chunks is the same correctness primitive that the present implementation enforces by reserving `max_pattern_len` $- 1$ warm-up bytes at every chunk boundary; this independent confirmation is particularly valuable because it derives from an entirely different parallel substrate, showing that the overlap is not just an artifact of thread-level parallelism. Likewise, DFA partitioning provides a precedent for splitting a pattern set across independent execution units, although the present work partitions the *input* rather than the automaton, leaving the goto table intact for shared, read-only access by all Pthreads workers. The principal divergence is the hardware target: PPS realizes parallelism through replicated pipeline stages and TCAM/SRAM primitives in a switch ASIC, whereas the present design pursues coarse-grained data parallelism on commodity multi-core CPUs, accepting lower theoretical throughput in exchange for portability and exact matching.

3.3.6 Practical Multi-Pattern Matching Approach for Fast and Scalable Log Abstraction

Tovarňák [23] addresses the throughput limitations of multi-pattern matching in log-abstraction pipelines, where each incoming log line must be classified by matching it against a potentially large set of regex-like patterns. The author observes that the naive strategy of iterating over the entire pattern set per log line incurs $O(|\text{patterns}|)$ cost

per input and degrades severely once the dictionary contains more than a few tens of entries, while general-purpose multi-regex libraries such as Google RE2 hit memory-cap limits when the DFA representation exceeds a few hundred patterns. The work is positioned alongside Aho-Corasick as a multi-pattern matcher built on a trie-based automaton, but it explicitly targets the same parallelism model adopted by the present thesis: a single automaton constructed once and shared, read-only, across multiple concurrent workers on a commodity multi-core CPU.

To recover throughput, the author proposes REtrie, a compressed trie that combines literal-character pointers with regex *token* pointers (named wildcards) and supports controlled backtracking only over the token segments, preserving an $O(|\text{input}|)$ matching cost in the typical case. Three design choices map directly onto the present Pthreads design: the trie is constructed sequentially and then declared immutable, so that worker processes need no synchronization during the matching phase; node arrays are compressed with PATRICIA-style path compression and two-level adaptive nesting, lowering the memory footprint of the otherwise sparse 256-entry pointer arrays and improving cache locality; and data-level parallelism is realized by routing disjoint subsets of input log lines to independent Erlang OTP processes, each of which traverses the same shared automaton without any inter-process locking. The system is implemented in Erlang with native-code generation via the HiPE compiler, so the parallelism model is process-based rather than thread-based, but the architectural invariant – read-only shared automaton, thread-local results, no locks on the hot path – is the same one adopted by the Pthreads design of the present thesis.

The evaluation is conducted on a dual-socket Intel Xeon E5-2650 v2 (2.60 GHz) with 64 GB of RAM, scaled from one to eight cores. The pattern sets include a hand-curated set of 22 syslog `auth/authpriv` patterns (P_{22}) and large Apache Hadoop sets (R_x) with up to 3,500 entries derived from source-code analysis. The runtime workloads include D_0, a real-world capture of 100 million syslog lines collected over one month, plus partially synthetic derivatives (D_x, H_x) of 1.1 million messages each constructed by controlled duplication and shuffling. Each measurement is averaged over ten runs with extreme values discarded. The single-core comparison shows that REtrie's running time remains essentially constant as the pattern count grows from 1 to 22, whereas the naive baseline reaches roughly 12 s on P_{22} versus ≈ 3 s for REtrie and RE2; for $R_x \geq 2{,}000$ patterns the Erlang REtrie even surpasses C++ RE2, which collides with its default DFA memory limit. The multi-core scaling on D_0/P_{22} exhibits a near-ideal $2\times$ speedup at two cores; at eight cores the 100-million-line workload is processed in 52.543 s, equivalent to $\approx 1{,}903{,}203$ log lines per second, with the speedup growth flattening past four cores, in line with the ceiling expected from Amdahl's law [12] and from memory bandwidth.

Taken together, the paper provides independent empirical validation of two design

choices that underpin the present Pthreads-based implementation of Aho-Corasick. First, the architectural invariant of a read-only shared automaton accessed concurrently by multiple worker processes is shown to deliver near-ideal scaling at low core counts on commodity multi-core hardware, with the diminishing-returns regime located around four to five cores, an observation consistent with the early plateau found in the present experiments. Second, the use of path compression and adaptive node sizing to improve cache locality reinforces the importance of the flat `goto_tbl` layout employed by the present implementation, even though the algorithmic context differs. Two limitations restrict the direct transferability of the results. The REtrie automaton is not an Aho-Corasick DFA: it lacks the failure and dictionary-suffix functions and processes each log line as a single bounded record, so the chunk-boundary overlap problem that motivates the present implementation's `max_pattern_len` $-$ `1` warm-up region simply does not arise here. Moreover, Erlang OTP scheduling abstracts away the cache, false-sharing and NUMA effects that the Pthreads design must address explicitly. The Tovarňák study therefore serves as a precedent for the parallel *strategy* (lock-free shared immutable automaton) rather than for its concrete realization, and as a useful throughput reference point on comparable Xeon-class hardware for multi-pattern matching at the line-of-text granularity.

3.3.7 Pattern Matching in YARA: Improved Aho-Corasick Algorithm

Regéciová et al. [24] address a particular failure mode of the canonical Aho-Corasick implementation embedded in the YARA rule-matching framework, a representative production scanning tool that relies on this algorithm. The authors observe that YARA's preprocessing phase extracts short literal substrings – termed *atoms* – to seed the AC automaton, but the atom-selection heuristic fails to represent the wildcards, character classes and metacharacters that increasingly appear in industrial signatures. As a result, the scanner either selects empty atoms, in which case the matching loop degenerates into a naive byte-by-byte comparison, or it issues a "slowing down" warning that renders the rule unusable on large-scale platforms such as VirusTotal Hunting. The bottleneck is therefore not in raw matching throughput but in the construction-time choice that determines which literal fragments seed the AC automaton.

To overcome this limitation, the authors propose MYARA, a fork of YARA 4.0.2 introducing three coordinated changes to the preprocessing phase while leaving the matching loop intact and sequential. First, atoms are encoded as 256-bit *bitmaps*, where each bit indicates whether the corresponding byte value is admissible at that position; a literal character, the dot metacharacter and a class such as `[a-z]` are therefore expressed in a uniform structure that supports fast subset and inclusion tests through bitwise operators. Second, the AC `goto` function is generalized to accept *sets* of characters per transition rather than single bytes, and the resulting non-determinism is resolved by

an explicit determinization step before the failure function is computed. Third, the existing Aho-Corasick Interleaved State Matrix (ACISM) – a flat array of 32-bit integers that encodes both `goto` transitions and failure links for $O(1)$ lookup – is updated to accommodate the denser slot occupancy produced by multi-symbol transitions. Because the resulting automaton remains entirely read-only after construction, the same shared-memory invariant relied upon by the present Pthreads design continues to hold.

The evaluation is conducted on a server equipped with an Intel Xeon E5-2697 v4 at 2.30 GHz running Debian 4.9.168, with binaries built using GCC 8.3.0. Three pattern dictionaries of contrasting composition are used: a hand-crafted set of five YARA rules targeting Bitcoin addresses, e-mail addresses and the INI format; the publicly available ReversingLabs corpus comprising 371 rules with 0% regular expressions; and the proprietary Avast set of 13,026 rules, of which only 2.1% are regular expressions. All workloads are scanned against an 8.54 GB real-world corpus that aggregates Bitcoin transaction history, e-mail archives and INI configuration files; each reported measurement is the minimum of fifteen daily runs collected over six days. On the degenerate Bitcoin regex (v1) the scan time falls from 169.700 s under canonical YARA to 17.623 s under MYARA, i.e. a $\approx 9.6\times$ speed-up that recovers the rule from the unusable regime. On the realistic ReversingLabs ruleset, where atoms are dominated by literal and hexadecimal strings, scanning time is essentially unchanged ($1{,}503.65$ s vs $1{,}504.90$ s); on Avast the global improvement is approximately 1%. Throughput in MB/s and memory consumption are not reported.

On balance, the paper offers two contributions that bear directly on the present Pthreads-based work even though it introduces no parallelism of its own. First, it documents and characterizes the ACISM representation that the present implementation – and any other shared-memory parallel Aho-Corasick variant – can adopt as a strictly read-only substrate, with the additional benefit that bitmap-encoded transitions remain compatible with the flat `goto_tbl[state * 256 + byte]` layout adopted in this thesis. Second, it provides documented single-threaded baselines on a 8.54 GB realistic corpus that serve as external reference points for the present evaluation. Three limitations restrict the direct applicability of the results, however: the optimization is concentrated in the construction phase, whereas the thesis specifically targets the matching phase; the reported gains evaporate on rulesets dominated by literal patterns (the ReversingLabs case), which closely resemble the literal-dominated Snort and Emerging Threats rule sets exercised in the present evaluation; and the absence of throughput measurements in MB/s precludes a direct quantitative comparison. The Regéciová et al. study therefore complements rather than competes with the present work: it improves the automaton fed to the scanner, while this thesis improves the parallel execution of the scanner against an unchanged automaton.

3.3.8 SIMD-Accelerated Regular Expression Matching

Sitaridi, Polychroniou and Ross [7] focus on the sequential bottleneck that arises when a DFA-based pattern matcher is embedded in an in-memory database column-filter operator: the per-character transition $s_{t+1} = \delta(s_t, c_t)$ creates a tight data dependency that defeats straightforward SIMD auto-vectorization, leaving most lanes of a wide vector register idle. The authors explicitly note that Aho-Corasick is equivalent to a minimal DFA once the failure transitions are absorbed into the transition table, so the analysis of vectorized DFA traversal performance transfers directly to the matching phase of any AC engine. Their focus is therefore the same matching-phase throughput targeted by the present thesis, but addressed at a different level of the parallelism hierarchy.

To exploit the unused lanes, the paper introduces a non-lockstep, short-circuit SIMD-DFA traversal algorithm. Rather than advancing all W lanes of the SIMD register in unison one character at a time – which wastes work whenever a string is rejected or accepted early – the algorithm tracks an independent input offset and DFA state per lane, uses SIMD *gather* instructions to fetch the relevant byte for each lane in a single instruction, and replaces any lane that reaches a sink state with a fresh input through selective loads. Three auxiliary techniques sustain the throughput: an eight-byte buffered gather amortizes the cost of non-contiguous loads, a two-way unrolled inner loop hides instruction latency, and a branch-free selective store records the identifiers of accepted strings. Critically for the present discussion, the authors compose this SIMD-level data parallelism with thread-level parallelism by spawning one worker per hardware thread; the DFA transition table is shared, read-only, across all workers, and string offsets are partitioned among them. This is structurally identical to the present Pthreads design, with the difference that the thesis partitions a *single* large input into chunks, whereas the paper partitions a column of many short strings.

The evaluation contrasts two platforms: an Intel Xeon E3-1275v3 with four Haswell cores and 256-bit AVX2, and an Intel Xeon Phi 7120P with 61 cores and 512-bit SIMD. Pattern dictionaries include a 90-state URL-validation regex (23 KB transition table), a 9-state e-mail regex, and synthetic multi-pattern dictionaries built from random English words whose DFA size is swept from 10 to 10^5 states to cross the L1, L2 and last-level cache boundaries. Inputs are synthetically generated string columns of varying length, selectivity and average failure point. Throughput is reported in GB/s of total bytes scanned together with the fraction of peak DRAM bandwidth achieved. The non-lockstep algorithm yields up to $2\times$ over a scalar baseline on the Haswell CPU and up to $5\times$ on the Xeon Phi, with the gap over lockstep reaching $3\times$ for long strings that reject early. Two further observations are particularly relevant. First, throughput scales linearly with hardware-thread count on both platforms, confirming that SIMD-level and thread-level parallelism compose without contention as long as the DFA is shared read-

only. Second, performance peaks when the DFA fits in the L1/L2 cache: once the transition table grows past last-level cache, vectorization gains shrink, with throughput reverting to a memory-bandwidth-bound regime.

Ultimately, three findings carry directly into the present Pthreads-based work, with the caveat that the underlying parallelism mechanisms differ. First, the explicit observation that AC is equivalent to a minimal DFA, combined with the demonstrated linear scaling of thread-level parallelism over a shared read-only transition table, independently validates the architectural choice of immutable post-construction `goto_tbl` central to the present design. Second, the empirical demonstration that cache residency dominates throughput once the DFA exceeds last-level cache provides a documented mechanism behind the saturation that the present thesis observes on large rule sets, motivating both the cache-locality concerns of the flat `goto_tbl[state * 256 + byte]` layout and the evaluation of memory-bandwidth contention as the thread count grows. Third, the orthogonality of SIMD-level and thread-level parallelism documented in this paper makes it a natural reference for positioning the Pthreads strategy as a baseline that future work could combine with SIMD intrinsics in the inner loop. Two limitations restrict the transferability of the numerical results: the workload is restricted to fixed-length strings in database columns, in contrast to the variable-length chunked file scans targeted by the present thesis, and the cache-residency assumption holds only for the small URL regex used in most experiments and breaks down for the large multi-pattern dictionaries that motivate AC in security applications.

3.3.9 Harry: A Scalable SIMD-based Multi-literal Pattern Matching Engine for Deep Packet Inspection

Xu et al. [25] tackle the per-byte cost of multi-literal pattern matching in software Deep Packet Inspection (DPI), which the authors identify as the dominant bottleneck of production stacks such as Snort, Suricata, ModSecurity and CloudFlare's WAF. The canonical Aho-Corasick (AC) algorithm is acknowledged as the traditional choice but is shown to leave large performance margins on the table compared with the SIMD-accelerated FDR engine of Hyperscan; FDR itself, however, requires three SIMD operations per input character regardless of the vector width, attains only 50% SIMD-lane utilization, and depends on the `VPSLLDQ` shift instruction whose 128-bit lane boundary makes AVX-512 unusable. The paper therefore poses the question of how far the throughput ceiling of single-threaded multi-literal matching can be raised on commodity x86 CPUs by aggressively redesigning the SIMD kernel, establishing a useful upper bound against which the multi-core Pthreads strategy of the present thesis can be positioned.

Three coordinated techniques constitute the Harry engine. A column-vector-based shift-or matching algorithm replaces FDR's row-vector decomposition: instead of load-

ing one row per input character, Harry loads one column vector per literal-position slot, so that the number of SIMD operations becomes proportional to the literal length ℓ (fixed at eight in the implementation) rather than to the input length, yielding only 0.41–0.71 operations per input character on AVX-512 and raising vector utilization to 87.5%. Two mask-table encodings – a compression-based variant (*Harry6b*, optimal for small rule sets) and a decomposition-based variant (*Harry12b*, optimal for large rule sets) – are selected at compile time by a heuristic based on the zero-bit rate of each encoding, keeping the column vector within 512 bits in both regimes. Finally, the legacy `VPSLLDQ` shift is replaced with a `VPERMB`-based shuffle, eliminating cross-lane data loss and enabling the kernel to exploit AVX-512 without algorithmic regressions. The matching pipeline preserves Hyperscan's two-stage structure – approximate filter followed by exact verification – so the read-only nature of the underlying tables remains intact, consistent with the lock-free shared-automaton invariant adopted by the present Pthreads design.

Harry is evaluated on an Intel Xeon Gold 6348 (2.60 GHz, two sockets of 28 cores each, AVX-512) running Ubuntu 20.04, with GCC 9.4.0 used without optimization flags. Literal rules are extracted from two production rule sets – OWASP ModSecurity Core Rule Set v3.3.2 and Snort Emerging Threats v2.9.0 – and partitioned into nine subsets spanning 10 to $3{,}000$ rules. Three input traces are used: 121 MB of real IXIA HTTP packets, 4 MB of Alexa non-HTTP packets and 763 KB of synthetic random bytes; each experiment is averaged over $1{,}000$ runs. The reported throughput ranges from 30 to 70 Gbit/s and represents a $26\times$–$45\times$ speed-up over canonical Aho-Corasick on real HTTP and non-HTTP traffic, peaking at $52\times$ on the random byte stream, and a $1.06\times$–$2.09\times$ improvement over FDR. The heuristic encoding selector picks the better variant in every evaluated configuration, and Harry maintains its advantage across the full 10–$3{,}000$ rule range, in contrast to the prior Teddy engine, which degrades sharply beyond roughly sixty rules.

Overall, the paper has two effects on the related-work positioning of the present thesis. First, it concretely quantifies the ceiling that single-threaded SIMD literal matching can reach on commodity AVX-512 hardware – 30–70 Gbit/s at 0.41 operations per character with 87.5% vector utilization – and reports that this regime is up to $52\times$ faster than the canonical Aho-Corasick algorithm. This establishes Harry as the contrast point against which a coarse-grained Pthreads parallelization of AC must justify itself, and clarifies that the rationale of the present work is not to outperform Harry on a single thread but to exploit multi-core resources where the SIMD path is already saturated, since the same input can be partitioned across cores and the AC automaton kept fully shared and read-only. Second, the paper documents that SIMD-level and thread-level parallelism are architecturally orthogonal: Harry runs entirely within a single thread, so its gains compose multiplicatively with any data-parallel chunking strategy on multi-core

CPUs. Two caveats restrict the direct numerical comparability of the results: the AC baseline is compiled by GCC without optimization flags while the Harry path is hand-vectorized, which likely inflates the reported $52\times$ ratio, and the engine replaces rather than parallelizes the Aho-Corasick automaton, departing from the strict-AC requirement of the present thesis. Harry should therefore be cited as the canonical ceiling for single-threaded literal matching in DPI, not as a parallel AC variant.

3.3.10 Characterizing Realistic Signature-based Intrusion Detection Benchmarks

Aldwairi, Alshboul and Seyam [6] address a methodological gap that affects every empirical study of pattern matching for signature-based Intrusion Detection Systems (NIDS), including the present thesis: there is no agreed standard for the datasets, the metrics or the simulation environment under which Aho-Corasick and competing algorithms are evaluated, and the resulting heterogeneity makes cross-paper comparisons unreliable. The authors review the most cited public datasets – DARPA, KDD Cup 99, NSL-KDD, Kyoto 2006+ and CAIDA – and document that all of them were designed for anomaly-based detection and therefore lack representative attack signatures; privately collected traces, conversely, cannot be redistributed for legal reasons. The paper therefore positions itself not as a parallelization contribution but as the construction of a reproducible NIDS benchmark stack that subsequent works – this thesis included – can adopt to ground throughput and speed-up claims in realistic conditions.

The contribution is delivered as a five-part artifact. First, a GUI-based parser ingests heterogeneous IDS rule formats and extracts the `content` and `uricontent` keywords of 120 Snort rule files, yielding hexadecimal-encoded signature dictionaries of varying cardinality (from 300 to over 2,400 signatures, suitable for scalability sweeps). Second, eight representative traces are curated from the 78 DEFCON17 Capture-the-Flag captures: four “good” traces (audio-video streaming, downloads, Hotmail, live chat) representing baseline traffic, two “ugly” traces (51 and 58) where 45% and 43% of packets carry attack signatures, and two “bad” traces (1 and 22) with intermediate 16%–17% malicious densities. Third, a pluggable matching engine implemented as a Windows DLL allows researchers to substitute their own algorithm under identical input and instrumentation. Fourth, four classical algorithms – Aho-Corasick, Knuth-Morris-Pratt, Wu-Manber and Rabin-Karp – are reimplemented strictly sequentially against the same engine to provide single-thread reference numbers. Fifth, an explicit set of metrics is fixed: runtime in microseconds, memory in bytes, and count of matched attack signatures per trace, each averaged over ten repetitions.

The hardware platform is not specified; the engine is implemented in C# and the authors flag that the runtime’s garbage collector limits the precision of memory measurements. The reported numbers are nevertheless useful as a sequential baseline. Aho-Corasick is the slowest of the three viable algorithms on the large “audio-video” trace,

taking $2{,}374{,}992$ µs against Wu-Manber's $619{,}469$ µs, and consumes the largest memory footprint, peaking at $6{,}840{,}694$ bytes versus $2{,}694{,}016$ bytes for KMP. The runtime-versus-signature-count sweep on the same trace shows that AC scales non-linearly: the time grows from $168{,}079$ µs at 300 signatures to $2{,}374{,}992$ µs at the full dictionary, an effect the authors describe as "almost exponential". The memory-versus-packet-count sweep on the "ugly" trace 58 reports a similar growth – from $29{,}402$ bytes at $25{,}000$ packets to $3{,}871{,}408$ bytes for the full trace – although this expansion most likely reflects accumulated match-record storage and C# allocation behavior rather than an intrinsic growth of the AC automaton, whose DFA trie is fixed by the signature set.

In sum, the paper provides the present thesis with two contributions that no other reviewed work delivers in combination. First, the DEFCON17 trace selection and the Snort-derived signature dictionaries are documented, redistributable benchmark inputs that can be replayed against the present Pthreads implementation to expose the multi-core speed-up under the same workload conditions used elsewhere in the literature. Second, the per-trace sequential AC runtimes – in the order of tens of milliseconds for the smaller traces and a few seconds for the largest – furnish numerical reference points against which the present multi-thread throughput can be situated. Three limitations restrict the direct use of the published numbers. The hardware on which they were measured is not specified, so absolute runtimes cannot be reproduced. The implementation language (C#) and its garbage collector make memory figures imprecise. And the paper does not report throughput in MB/s or Gbps, which are the metrics the present thesis adopts; conversion from microseconds plus payload size in bytes is therefore required when quoting the Aldwairi figures as competitors. The paper should be cited for benchmark methodology and realistic-workload selection rather than as a parallel-AC technique, and it complements the multi-core, multi-threaded contributions reviewed above by anchoring the evaluation in the canonical NIDS context that motivates Aho-Corasick parallelization in the first place.

3.4 SYNTHESIS AND IDENTIFIED GAP

The systematic review answered both research questions by identifying the predominant parallelization strategies for Aho–Corasick on multi-core CPUs and the optimization techniques most frequently employed in practical applications. Read together, the ten primary studies cover a broad design space but leave a specific combination unaddressed. The reviewed parallelizations either target non-CPU substrates (GPUs in Thambawita et al. [5], FPGA fabric in Deyannis et al. [21], and programmable switch ASICs in Jepsen et al. [22]); rely on a coordination model that is not native shared-memory threading (MPI in Qu et al. [20], Erlang OTP processes over a non-Aho–Corasick automaton in Tovarňák [23]); restructure or replace the classical automaton rather than parallelize it unchanged (the head–body decomposition of Lee and Yang [2],

the SIMD literal engines of Sitaridi et al. [7] and Xu et al. [25]); or optimize the construction phase while leaving matching sequential (Regéciová et al. [24]). Aldwairi et al. [6] contributes benchmark methodology rather than a parallel technique. Recurrently, the studies that do measure shared-memory scaling report that throughput saturates once the automaton's transition table escapes the last-level cache, yet none isolates and characterizes that memory-bound regime as the primary object of study.

The gap addressed by the present work is therefore the following: a portable, POSIX Threads parallelization of the *unmodified* Aho–Corasick automaton on commodity shared-memory multi-core CPUs, based on input-text partitioning with a $\texttt{max\_pattern\_len} - 1$ overlap and a strictly read-only transition table, evaluated specifically in the regime where the automaton greatly exceeds the last-level cache and with exact-match equivalence to the sequential baseline. No reviewed study delivers this combination. Based on these findings, the next chapter presents the design and implementation of the proposed shared-memory parallel version of the Aho–Corasick algorithm.

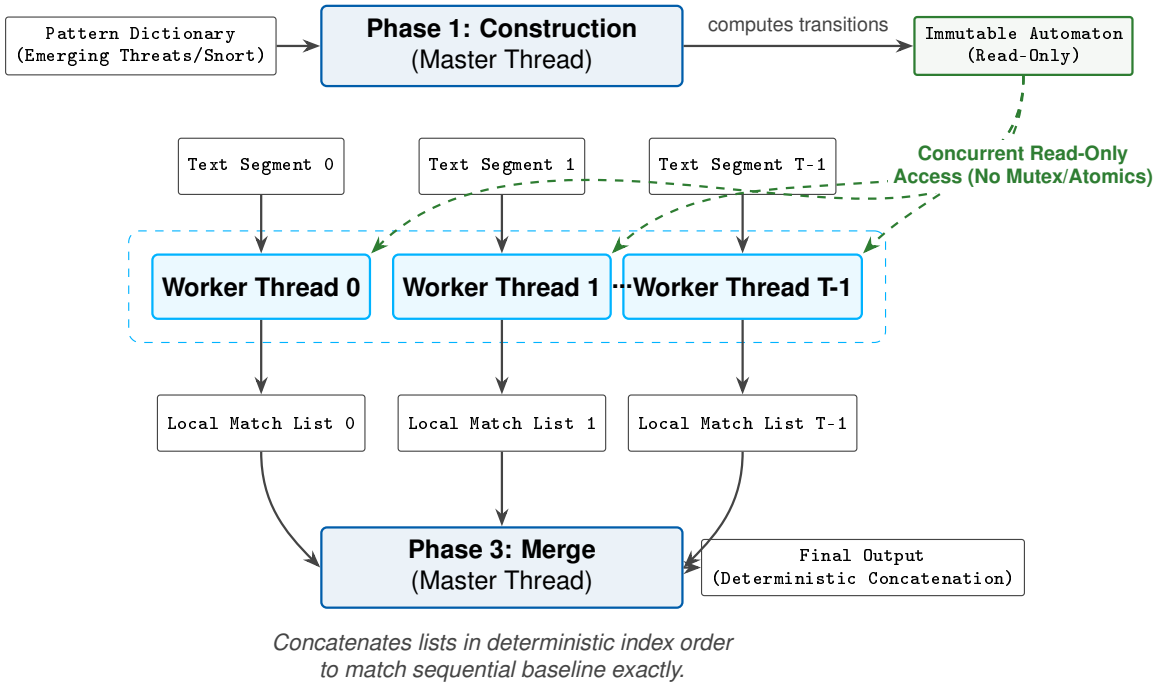

4

4 PROPOSED SOLUTION

This chapter presents the proposed solution: a modular parallel framework for the Aho-Corasick multi-pattern matching algorithm, targeting shared-memory multi-core CPUs. The design is guided by a single structural decision from which every other property follows. The default execution is split into separate phases: the master thread builds the automaton, after which its state-transition tables are declared strictly read-only. An optional construction variant parallelizes the failure-link traversal, but in every case the completed automaton is immutable before scanning begins. Because the automaton is never mutated during scanning, multiple worker threads can traverse it concurrently without any mutual-exclusion primitive on the hot path. This lock-free property is what allows the parallel search to approach the limits imposed by the hardware rather than by software contention.

The framework is organized so that distinct optimization ideas can be plugged in and measured independently. Two complementary forms of scan parallelism are considered: partitioning the *input text* among threads, and partitioning the *pattern dictionary*. In addition, an algorithmic optimization of the match-emission step is proposed that benefits both the sequential and the parallel cases, and a build-phase optimization is evaluated separately. The remainder of this chapter describes each component. The overall phased execution flow of the proposed parallel framework is illustrated in Figure 5.

Figure 5: Parallel execution pipeline of the proposed framework, divided into three phases: sequential construction of the read-only automaton (Phase 1), concurrent parallel search across text segments (Phase 2), and sequential deterministic merge of match lists (Phase 3).



Source: Prepared by the author.

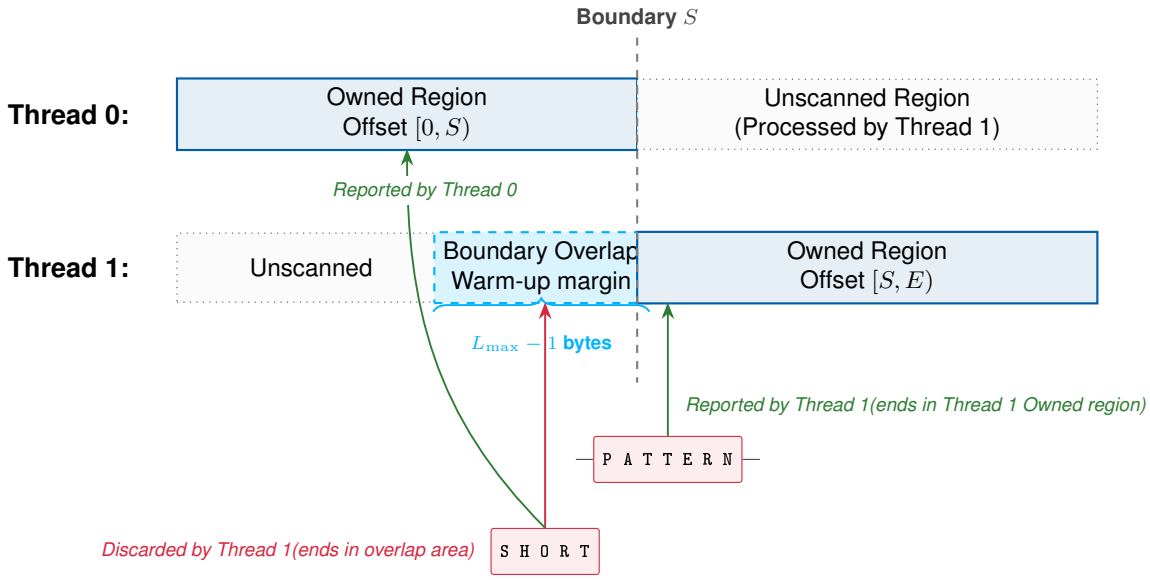
4.1 DATA PARTITIONING AND BOUNDARY OVERLAP

To exploit thread-level parallelism on a single large input, the framework partitions the text buffer into contiguous segments, one per worker thread. Each worker traverses the shared automaton over its own segment and records matches in a private list, so that no synchronization is required while scanning.

However, a naive partitioning of the byte stream would miss any pattern that crosses the boundary between two adjacent segments, because neither worker would observe the full pattern. To prevent these boundary false negatives, the design uses a controlled overlap. Let L_{max} be the length of the longest pattern in the dictionary. A worker responsible for the segment spanning offsets $[S, E)$ does not begin scanning at S but at $S - L_{\text{max}} + 1$, except for the first worker, which starts at offset 0. This warm-up margin guarantees that, by the time the worker reaches the start of its owned region at S, the automaton is in the same state it would have reached in a single sequential pass from offset 0, so every boundary-crossing pattern is observed by exactly one worker. Any match whose end position falls before S is suppressed, because it is owned and reported by the previous worker. Conversely, assigning each match to the segment

containing its end position keeps ownership disjoint. This preserves an exact one-to-one correspondence with the sequential baseline and avoids duplicate detections. An overlap of $L_{\text{max}} - 1$ is the minimum that achieves this property; any smaller value loses matches, and any larger value only adds redundant work. This boundary-crossing detection and overlap logic is illustrated in Figure 6.

Figure 6: Data partitioning and boundary overlap scanning layout, showing matching ownership and warm-up margins across Thread 0 and Thread 1.

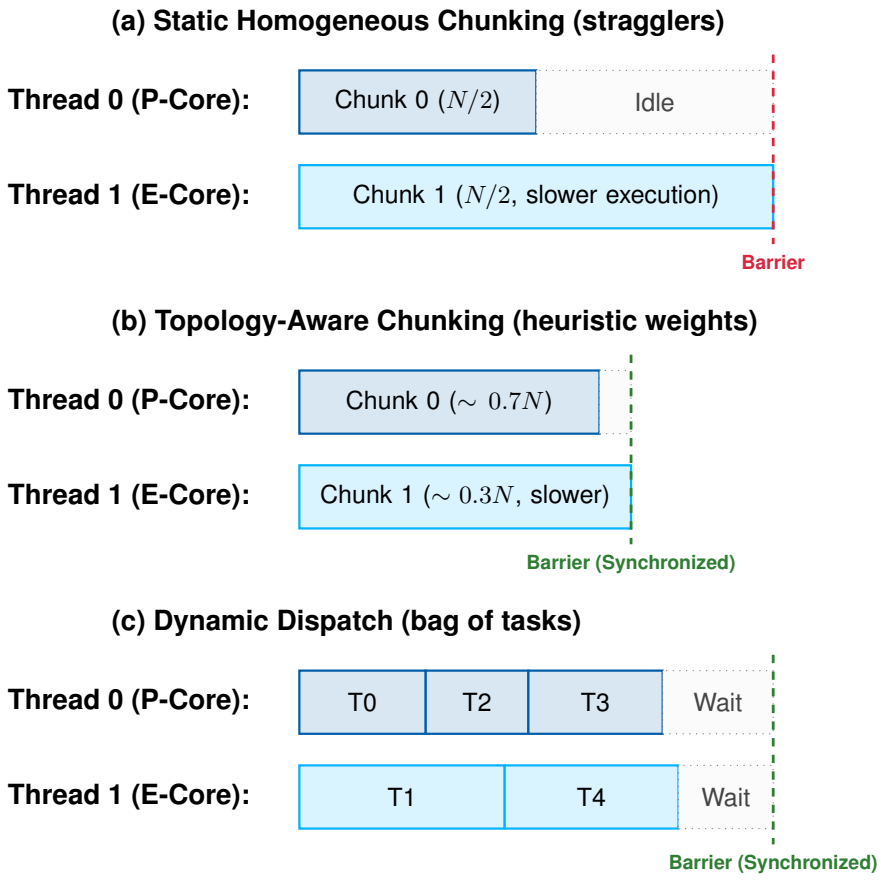


Source: Prepared by the author.

4.2 LOAD BALANCING ON HETEROGENEOUS CORES

Uniform segments are optimal only when all cores are equally fast. On a hybrid processor, where Performance and Efficiency cores run at different clock frequencies, equal-sized segments cause the threads on slower cores to finish late, leaving the faster threads idle at the final synchronization barrier. The proposed framework therefore includes a topology-aware partitioning strategy that detects the heterogeneous layout and scales each segment in proportion to the relative speed of the core processing it. The goal is to equalize the finishing times of all workers, reducing the idle time wasted at the barrier. As an alternative that requires no knowledge of the topology, the framework also supports dynamic dispatch, in which the text is divided into many small tasks that idle workers claim through a single atomic counter, so that faster cores naturally process more tasks. Figure 7 contrasts the three partitioning policies and the idle time each leaves at the barrier.

Figure 7: Partitioning policies on heterogeneous cores. Bar length represents elapsed execution time, while labels indicate assigned text size or task identity: (a) static homogeneous chunking leaves the faster Performance core idle at the barrier (stragglers); (b) topology-aware, frequency-weighted chunking sizes each segment to its core's speed; (c) dynamic dispatch hands out small tasks on demand. Both (b) and (c) equalize finishing times.



Source: Prepared by the author.

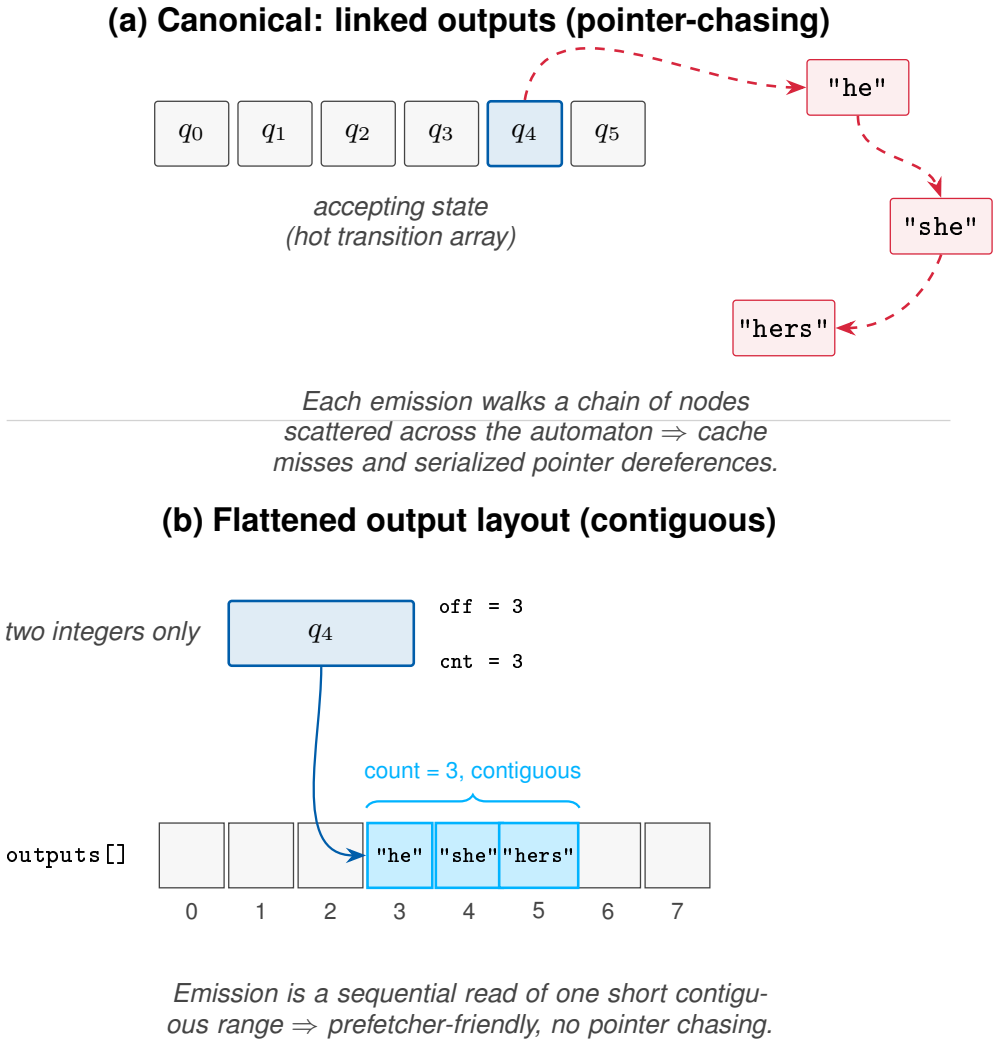
4.3 FLATTENING THE MATCH-EMISSION PATH

In the canonical Aho-Corasick algorithm, each accepting state references the patterns that end there through linked structures, and reporting all matches at a given state requires walking these pointer chains. With a large dictionary, these pointers are scattered across an automaton much larger than the cache. As a result, each emission causes cache misses and serialized pointer dereferences that compete with the hot state-transition traffic.

To remove this cost, the design proposes a flattened output layout. During con-

struction, the patterns associated with every state are collected into a single contiguous array, and each state stores only a starting offset and a count into that array. At scan time, emitting the matches of a state becomes a sequential read of a short, contiguous range. The hardware prefetcher handles this efficiently, removing pointer dereferences from the inner loop. This optimization is purely a memory-layout change: it preserves the exact set of reported matches and is inherited by every scanning variant, whether sequential or parallel. Figure 8 contrasts this contiguous layout with the canonical linked list structure.

Figure 8: Comparison of match-emission layouts: (a) canonical linked outputs using pointer-chasing across scattered memory nodes, and (b) proposed flattened output layout storing contiguous pattern indices.

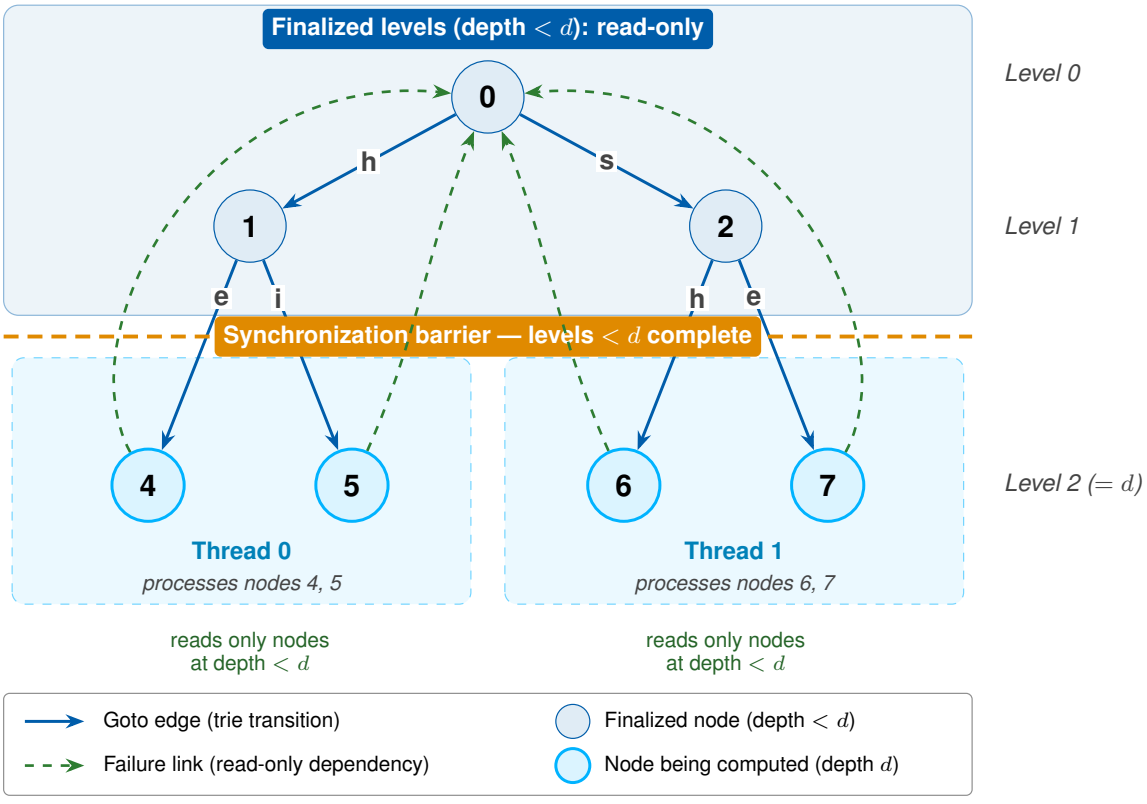


Source: Prepared by the author.

4.4 PARALLEL CONSTRUCTION OF THE AUTOMATON

The scan-phase strategies above assume that the automaton has already been constructed. The framework also includes a build-phase variant that parallelizes the breadth-first traversal used to compute failure links. Trie insertion remains sequential, because each pattern may create or reuse shared prefix nodes. After the trie exists, however, the failure link of a state at depth d depends only on links from shallower depths. All states at the same breadth-first level can therefore be processed concurrently, followed by a barrier before the next level begins. The barrier preserves the dependency order of the sequential construction while exposing the independent work inside each level. Figure 9 illustrates this level-synchronous structure.

Figure 9: Level-synchronous parallel construction of the failure function. Threads compute the failure links of all states at one breadth-first level in parallel, reading only already-finalized links from shallower levels (green), and meet at a barrier before advancing to the next level.



Source: Prepared by the author.

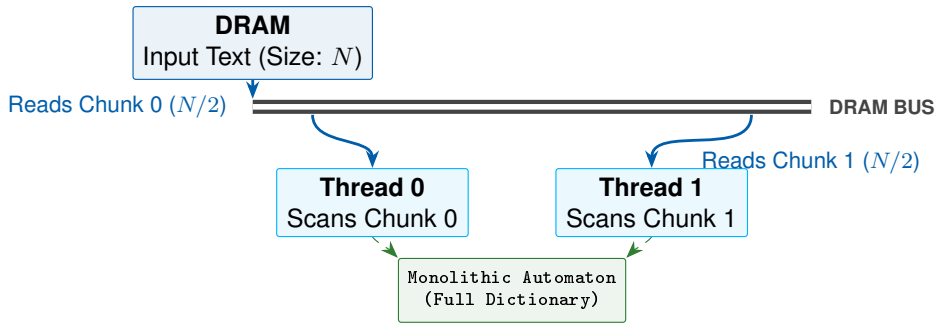
4.5 PARTITIONING THE DICTIONARY

All the strategies above partition the text while keeping the dictionary intact. A complementary approach, evaluated for contrast, partitions the dictionary instead. In this approach, the pattern set is divided into K groups based on the first byte of each pattern, and a smaller sub-automaton is built for each group. Every sub-automaton is

then scanned over the input. The motivation is that each sub-automaton has a smaller working set, which might fit better in the cache and reduce the memory pressure of a single monolithic automaton. The framework also supports a two-dimensional composition that combines dictionary sharding with text chunking. In that composition, each shard is scanned in parallel over chunks of the input, but every byte of the text must still be processed once per shard. Pure text chunking therefore reads each byte once, whereas sharding with chunking multiplies text traffic by the number of shards while reducing the automaton footprint seen by each shard. Whether the locality gained by smaller sub-automata compensates for this repeated input traffic is an empirical question, answered in the results chapter. Figure 10 summarizes this traffic difference.

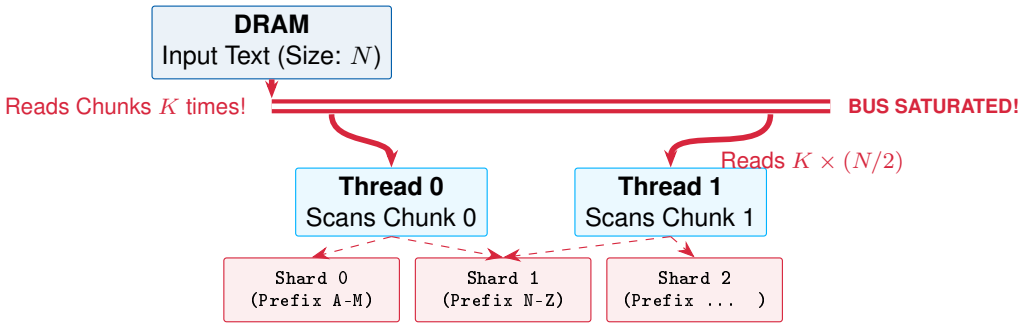
Figure 10: Text-reading traffic in the two partitioning designs: (a) pure text chunking reads each input byte once while workers share the full automaton; (b) two-dimensional composition scans chunks against all K dictionary shards, repeating the input reads while reducing each sub-automaton.

(a) Pure Text Chunking (Text Parallelism)



Total Text Read: $1 \times N$
Each byte is read once from main memory.
DRAM bus operates under moderate load, avoiding saturation.

(b) Two-Dimensional Composition (Falsified Hypothesis)



Total Text Read: $K \times N$
Chunks must be scanned against all K dictionary shards.
Multiplies memory traffic, saturating the bus and degrading speedup.

Source: Prepared by the author.

5

5 METHODOLOGY

This chapter describes the experimental methodology used to evaluate the parallel Aho–Corasick scanning strategies introduced in the previous chapter. The evaluation targets three properties of each strategy — search performance, scalability with the number of threads, and correctness with respect to the sequential baseline — under a shared-memory multi-core environment and over pattern dictionaries and text corpora representative of signature-based intrusion detection. Because the workloads of interest produce automata whose footprint far exceeds the last-level cache, the reported figures are expressed as relative quantities (speedup, throughput, and parallel efficiency) rather than as absolute production capacity, so that the conclusions concern scaling behaviour instead of the peak rating of a single machine.

The chapter is organized in the order in which the reader needs the information. It first establishes the experimental environment and software stack; then characterizes the pattern dictionaries and text corpora and the role each one plays; states the three-phase execution model and the invariants that make the parallelization sound; enumerates the evaluated strategies and the axes along which they compose; separates the performance metrics from the supporting diagnostic instrumentation; defines the per-phase measurement protocol; describes how correctness was established before any performance was reported; and closes with the experimental design, stating each hypothesis, the experiment that tests it, and the section of the next chapter that reports the outcome.

5.1 EXPERIMENTAL ENVIRONMENT

The primary measurement platform, and the source of every headline number, table, and figure in this work, is a single Intel Core i5-1235U processor; the canonical data set is the formal sweep executed on it on 2026-05-29. The i5-1235U is a hybrid (heterogeneous) design of the Alder Lake generation: it integrates two Performance cores (P-cores) with simultaneous multithreading and eight Efficiency cores (E-cores) without it, exposing twelve logical hardware threads in total. The two core types differ in maximum clock frequency (approximately 4.4 GHz for P-cores and 3.3 GHz for E-cores), which is the source of the load-balancing problem addressed by the frequency-weighted partitioning strategy. The memory hierarchy comprises a private L1 data cache per core

(48 KiB on P-cores, 32 KiB on E-cores), 1.25 MiB of L2 per P-core (and 2 MiB per E-core cluster), and a single 12 MiB L3 cache shared by all cores. Main memory is 31 GiB of dual-channel DDR4, and the aggregate memory bandwidth — on the order of a few tens of GB/s for this configuration — is the shared resource for which the parallel scans ultimately contend.

The software stack is a purpose-built laboratory written in C11, compiled with GCC 13.3.0 using `-O3 -march=native` and an aggressive warning profile, and linked against the POSIX Threads library [26]. Experiments ran under Linux (kernel 6.17.0). To minimize scheduling and frequency-scaling artifacts, the CPU frequency governor was set to `performance` for the canonical measurement campaign; the topology-aware searchers (`pthread_chunked_v3` and `pthread_chunked_v3_flat`) additionally pin each worker thread to a dedicated logical CPU via `pthread_setaffinity_np`, while the remaining searchers rely on OS scheduling without explicit binding. The build was additionally validated under sanitizer instrumentation, namely the Address and Undefined Behavior sanitizers and the Thread Sanitizer, as discussed in Section 5.7.

Two further runs play strictly secondary roles and do not displace the canonical i5 sweep. The first is a portability run on an AMD Ryzen 9 9950X (Zen 5), a homogeneous design with sixteen identical cores and thirty-two threads (two-way simultaneous multithreading) and a non-unified 64 MiB L3 organized as two 32 MiB slices, one per core complex, backed by DDR5; its sweep, dated 2026-06-25, is used only to assess the external validity of the i5 findings and is reported as a compact portability comparison in the next chapter, never as a substitute for the canonical numbers. The second is a reproducibility re-run of the i5 sweep on 2026-06-28, executed headless on a cooler machine; it is used to characterize run-to-run variance and is likewise not a source of headline figures. The single-thread baseline reproduces to within about 3% between the two i5 runs, but the saturated twelve-thread peak speedups vary substantially across separate invocations — a limitation made explicit in the measurement protocol below.

5.2 PATTERN DICTIONARIES AND TEXT CORPORA

The workloads were chosen to sweep the automaton footprint across the cache hierarchy while keeping the input text realistic for the intrusion-detection domain. Table 2 summarizes the pattern dictionaries, which were derived from publicly available Snort and Emerging Threats rule sets, and Table 3 summarizes the text corpora.

Table 2: Pattern dictionaries, the resulting automaton footprint, and their experimental role.

Dictionary	Patterns	States	Footprint	Experimental role
Snort-100	100	1,939	1.93 MiB	Smallest automaton; lower-bound contrast regime.
Snort-1k	∼1,000	11,974	11.92 MiB	Intermediate footprint; brackets the throughput cross-over.
Snort (full)	4,188	55,479	55.23 MiB	Primary target dictionary (moderate footprint).
Emerging Threats	44,678	508,896	506.78 MiB	Largest footprint; most aggressive memory stressor.

Source: Prepared by the author from Snort and Emerging Threats rule sets.

Table 3: Text corpora used as scanning input.

Corpus	Size	Characterization and role
SimpleWiki	∼1.19 GiB	Natural-language encyclopedic text with low match density; used as the contrast corpus in the cross-corpus experiment, which isolates the effect of the text content.
Enron	∼1.32 GiB	Real e-mail messages with medium match density; the canonical benchmark corpus.
Enron ($8\times$)	∼10.59 GiB	The Enron corpus replicated eightfold; amortizes the build phase and reduces scheduler noise in the scaling curves.

Source: Prepared by the author. The Enron corpus derives from the public Enron Email Dataset.

The two smaller dictionaries (Snort-100 and Snort-1k) serve as *contrast regimes* that bracket the point at which the automaton stops fitting comfortably in cache; the full Snort dictionary and the Emerging Threats subset are the actual targets of the study, the latter being the most aggressive stressor, with an automaton far larger than the last-level cache of any platform used here. The footprints in Table 2 are reported as absolute, platform-independent sizes; how each footprint relates to a particular cache level depends on the machine and is therefore interpreted in the next chapter against the cache hierarchy of each platform (for the reference i5, the 12 MiB shared L3).

5.3 EXECUTION MODEL AND INVARIANTS

Every execution of the laboratory follows three strictly separated phases; this separation is the foundation of the lock-free design.

1. **Construction.** The patterns are inserted into a trie, and a breadth-first (BFS) traversal computes the goto, failure, and dictionary-suffix links, together with the precomputed output structures. The resulting automaton is then declared immutable.

2. **Search.** Each worker thread reads the shared, read-only automaton and its assigned portion of the input text, emitting matches into a private, thread-local list. No mutex or atomic operation is used on the hot path, because the automaton is never modified during this phase.

3. **Merge.** After all workers join, the master thread concatenates the thread-local lists in a deterministic order. This phase is always sequential and accounts for a negligible fraction of the total time, so it is not a target for optimization.

This study respects a set of non-negotiable invariants that make the parallelization sound. The automaton is strictly read-only during the search; the hot loop uses neither locks nor atomics; matches remain thread-local until the join; and the partitioning preserves match ownership without duplication. For any text-level partitioning, the minimum overlap between adjacent segments is $L_{\text{max}} - 1$ bytes, where L_{max} is the length of the longest pattern — as established in the previous chapter, the smallest margin that guarantees that a boundary-straddling pattern is detected by exactly one worker. Match ownership is made disjoint by assigning each boundary-crossing match to a single segment according to its end position.

5.4 EVALUATED STRATEGIES

The strategies are organized as a set of orthogonal design axes so that each performance effect can be attributed to a single cause. Every variant is a combination of one choice per axis, all selectable at run time within the same binary and all sharing the same automaton. The axes, and which choices are mutually exclusive versus composable, are summarized in Table 4.

Table 4: Evaluated strategies as orthogonal design axes; the work-distribution options are alternatives, while the emission and construction axes compose with any of them.

Axis	**Options (one per run; mutually exclusive within the axis)**
Work distribution	Sequential baseline; static homogeneous chunking; topology-aware weighted chunking; dynamic chunk dispatch (bag of tasks); dictionary sharding by pattern prefix; two-dimensional sharding $\times$ chunking.
Emission layout	Chained (pointer-chasing) *or* flattened contiguous output table; composes with any work-distribution choice.
Construction	Sequential build *or* level-synchronous parallel build (BFS); composes with any work-distribution choice.

Source: Prepared by the author.

Two of the three axes compose freely. The emission layout (chained pointer-chasing versus a flattened, contiguous output table) and the construction mode (sequential versus level-synchronous parallel build) can be paired with any work-distribution choice and with the sequential baseline, because the parallel build produces an automaton bit-for-bit identical to the sequential one and the emission layout does not change which matches are produced. The work-distribution axis, in contrast, is mutually exclusive: a run partitions the work in exactly one way. The named searchers in the laboratory are therefore compositions of these axes — for example, static chunking with the flat layout, or topology-aware chunking with the flat layout — which is precisely what allows one axis to be varied while the others are held fixed.

The work-distribution options differ in how they assign work to threads. Static homogeneous chunking splits the text into equal segments, one per thread, with the boundary overlap required for correctness. Topology-aware chunking and dynamic dispatch are two *distinct* answers to the load imbalance that equal segments cause on a heterogeneous processor, and they are not the same mechanism. Topology-aware chunking sizes each segment *statically*, in proportion to the maximum clock frequency of the core that will execute it — read once from the operating system's per-core frequency information — and pins the worker to that core, so that a faster P-core receives a proportionally larger segment before the scan begins. Dynamic dispatch instead splits the text into many small, uniform tasks that idle workers pull from a shared atomic counter *at run time* (a bag-of-tasks scheme), correcting imbalance reactively rather than from a hardware model. The dictionary-parallelism options partition the pattern set instead of the text: prefix sharding builds K independent sub-automata, and the two-dimensional

composition combines sharding with text chunking to test whether the two kinds of parallelism add. An exploratory software-prefetch variant of the chunked searcher also exists in the laboratory as a microarchitectural probe; it introduces no new axis and is excluded from the controlled grid and from the reported results.

5.5 METRICS AND INSTRUMENTATION

The performance metrics follow the standard definitions introduced in the theoretical framework. Search throughput is reported in MB/s (and, where useful, in Gbps); speedup is the ratio of sequential to parallel search time, as in Eq. 1; and parallel efficiency is the speedup normalized by the thread count, as in Eq. 4. Search time is the quantity these are derived from, and it is always measured separately from construction time, so that the cost of rebuilding the automaton — relevant to rule reloading — is never conflated with scanning throughput.

A second group of quantities does not measure performance but supports its interpretation, and is reported as diagnostic instrumentation rather than as a result. Per-thread timings and the coefficient of variation across repetitions measure load balance and stability, respectively: the per-thread spread exposes synchronization waste at the final barrier, while the coefficient of variation quantifies how noisy a measurement is rather than how fast it is. Match counts are recorded for every run as a correctness check, not a performance figure (see Section 5.7). Construction (build) time is recorded for the parallel-build experiment. Finally, the laboratory writes every run, together with a snapshot of its environment, to a structured log that is ingested into an SQLite database, which is the substrate for all aggregation and cross-checking; the automaton footprint in bytes is retained there too — not as a headline figure, but as the causal variable used to explain throughput across regimes.

5.6 MEASUREMENT PROTOCOL

Each reported data point follows the per-phase repetition protocol summarized in Table 5. Every phase begins with a number of *warm-up* iterations whose times are discarded: the warm-up pays the one-time costs of page faults and cache population so that the subsequent timed iterations measure steady-state behaviour. For each timed series both the mean and the best (minimum) time are kept; the mean is used for the reported throughput, and the dispersion between iterations is retained as the stability indicator described in the previous section.

Table 5: Per-phase repetition protocol of the formal sweep (2026-05-29), verified against `sweep.db`.

Phase	Description	Warm-ups	Timed iters
A	Speedup curves	2	5
B	Footprint sweep	2	5
C	Cross-corpus	2	5
D	Per-thread diagnostic	1	3
E	Parallel construction	0	1

Source: Prepared by the author from `sweep.db` (sweep 2026-05-29).

Phase A measures the speedup curves, the headline scaling result. It sweeps thread counts $T \in \{1, 2, 4, 6, 8, 10, 12\}$ over the canonical Enron corpus and $T \in \{1, 4, 8, 12\}$ over the eight-fold replica, running every parallel searcher against the two single-thread baselines, with two warm-ups and five timed iterations per point.

Phase B measures the automaton footprint against throughput. Holding the corpus fixed at Enron, it varies the dictionary across the four footprints of Table 2 to locate the throughput cross-over, again with two warm-ups and five timed iterations.

Phase C isolates the effect of the text content. Holding the dictionary fixed at the full Snort set, it compares the natural-language SimpleWiki corpus against Enron, with two warm-ups and five timed iterations, so that any throughput difference is attributable to the corpus rather than to the automaton.

Phase D is the per-thread diagnostic. At the full thread count it records per-thread timings for each parallel searcher — one warm-up and three timed iterations — to quantify barrier idle time and load balance.

Phase E measures parallel construction. It compares the sequential build against the level-synchronous parallel build across the four dictionaries; since only the build time is of interest, each build is timed once with no warm-up.

The peak speedups reported at the saturated thread count must be read with one limitation in mind. On the reference i5, which exposes only twelve hardware threads, repeated full sweeps reproduce the single-thread baseline to within about 3% but place the twelve-thread peak in visibly different regimes from one invocation to the next, an effect that the within-run coefficient of variation does not capture; the saturated peaks are therefore treated as run-dependent rather than exact. The reference platform is additionally a thermally constrained mobile part, whose sustained all-core clocks sit below its short-burst peaks, so the published values are the sustained, reproducible figures; the thermal behaviour itself is not used as a premise here and is revisited, where the data warrant, in the results and conclusion.

5.7 CORRECTNESS VALIDATION

Correctness is a precondition for any performance claim: a faster scan is worthless if it does not return exactly the matches that the sequential algorithm returns. The goal of every optimization here is to reduce execution time while preserving the output of the sequential baseline, so each parallel variant was required to reproduce that output before it was benchmarked. Correctness was established at three complementary levels, described in turn below.

First, at the level of the match *set*, the automated test suite requires every strategy to reproduce the exact set of matches of the sequential baseline — each match identified by its end position and pattern identifier — across thread counts of $\{1, 2, 3, 4, 7, 8\}$. The same suite verifies that the level-synchronous parallel construction yields an automaton bit-for-bit identical to the sequentially built one, so that swapping in the parallel build can never alter the reported matches.

Second, at the level of the full-scale *multiset*, the MD5 digest of the canonicalized match stream — the complete set of matches placed in a deterministic order by end position and pattern identifier — was computed for each parallel strategy on the production-scale workloads and compared against the digest of the sequential baseline. Agreement certifies the exact multiset of matches (their count, end positions, and pattern identifiers), a far stronger guarantee than match-count parity; it held for every strategy up to a thread count of 64, a $5.3\times$ oversubscription of the hardware, demonstrating that the partitioning logic is robust well beyond the physical thread count.

Third, at the level of segment boundaries and emission order, the strategies whose thread-local results are merged in text order (the static-chunking strategies) reproduce even the digest of the raw, un-canonicalized emission stream, confirming that every boundary-straddling match is assigned to exactly one segment. The dynamic-dispatch and dictionary-sharded strategies emit the identical multiset in a different order, so for them only the canonicalized digest coincides, which is the expected and correct behaviour.

Finally, ThreadSanitizer reported no data races during the parallel search, empirically confirming the read-only-automaton invariant and the absence of unsynchronized shared writes on the hot path.

5.8 EXPERIMENTAL DESIGN AND HYPOTHESES

The objective of this work is to accelerate multi-pattern matching by parallelizing Aho–Corasick on shared-memory multi-core CPUs. That objective is decomposed into a set of explicit hypotheses, each phrased so that an experiment can falsify it and each isolating a single design axis while the others are held fixed. Table 6 maps every hypothesis to the experiment that tests it and to the place in the next chapter where its outcome is reported; throughout, the comparison is against the sequential baseline for headline speedup and against the strongest text-parallel strategy when the question

concerns composition.

Table 6: Hypotheses, the experiment that tests each one, and where the outcome is reported.

Hypothesis	Experiment	Reported i
Text chunking scales sublinearly on a multi-core CPU but suffers load imbalance when segments are sized uniformly.	Speedup curves (Phase A)	Fig. 12
The automaton footprint, not the corpus content, governs throughput once the working set escapes the cache.	Footprint and cross-corpus sweeps (Phases B–C)	Table 7
The flattened output layout reduces emission cost in both the sequential and the parallel case.	Layout comparison	Table 8
Topology-aware weighting reduces barrier idle time relative to uniform chunking on heterogeneous cores.	Per-thread diagnostic (Phase D)	Table 9
A level-synchronous parallel construction reduces build time for large dictionaries.	Build sweep (Phase E)	Table 10
Prefix-based dictionary sharding recovers throughput in the memory-bound regime by shrinking the per-shard working set.	Sharding experiment	Table 11
A two-dimensional composition of sharding and chunking combines cache locality with text parallelism.	Composition experiment	Table 12

Source: Prepared by the author.

Each row answers a different question: how performance scales with threads, what governs throughput, whether the emission layout pays off, whether topology-aware balancing helps, whether construction parallelizes, and whether dictionary parallelism helps on its own or in composition. Several hypotheses anticipate negative or qualified outcomes, and reporting a strategy that fails to outperform a simpler counterpart is treated as a result in its own right, since it delimits where plausible optimizations stop paying off on this class of hardware. The next chapter presents the outcome of this plan, following the same order.


6

6 RESULTS AND DISCUSSION

This chapter presents the experimental evaluation of the proposed parallel Aho-Corasick implementation. Following the methodology described in the previous chapter, each experiment is reported in a fixed order—experimental question, configuration, the measured data, an objective description of the observed pattern, the interpretation that the data permit, and a local conclusion tied to the corresponding hypothesis—so that what was measured stays separate from what is inferred.

The chapter is organized as follows. It first characterizes how the execution platform responds to a growing automaton footprint, establishing the experimental context for the analyses that follow. The subsequent sections then evaluate the individual optimization axes introduced earlier—text partitioning and its scalability, the flattened output layout, load balancing on heterogeneous cores, parallel construction, dictionary sharding, and the two-dimensional composition of sharding with chunking—each as a controlled experiment that varies a single axis. The final section consolidates the findings and positions them against the related literature.

All search and construction figures reported here come from the same sustained measurement campaign on the i5-1235U platform; the end-to-end pipeline times are reconstructed from that same campaign. Every parallel strategy was first verified to reproduce the exact match set of the sequential baseline. Throughout, speedup is measured against the single-threaded sequential baseline, and the reported values are the sustained, under-load figures, which are conservative relative to cold-burst peaks.

6.1 IMPACT OF THE AUTOMATON FOOTPRINT ON THROUGHPUT

Before evaluating the proposed optimizations, this experiment characterizes how the execution platform responds to increasing automaton footprints. The question it answers is whether single-threaded scanning throughput is governed by the size of the automaton or by the content of the text. To isolate the footprint, the dictionary is swept from one hundred patterns to roughly forty-five thousand while the corpus is held fixed and a single thread is used, which removes any threading confound from the measurement. Table 7 reports the resulting throughput.

Table 7: Single-threaded scanning throughput as the automaton footprint grows (corpus fixed, single thread).

Dictionary	Patterns	Automaton	Throughput (MB/s)
Snort-100	100	1.93 MiB	483.1
Snort-1k	∼1,000	11.92 MiB	337.3
Snort (full)	4,188	55.23 MiB	240.7
Emerging Threats	44,678	506.78 MiB	125.6

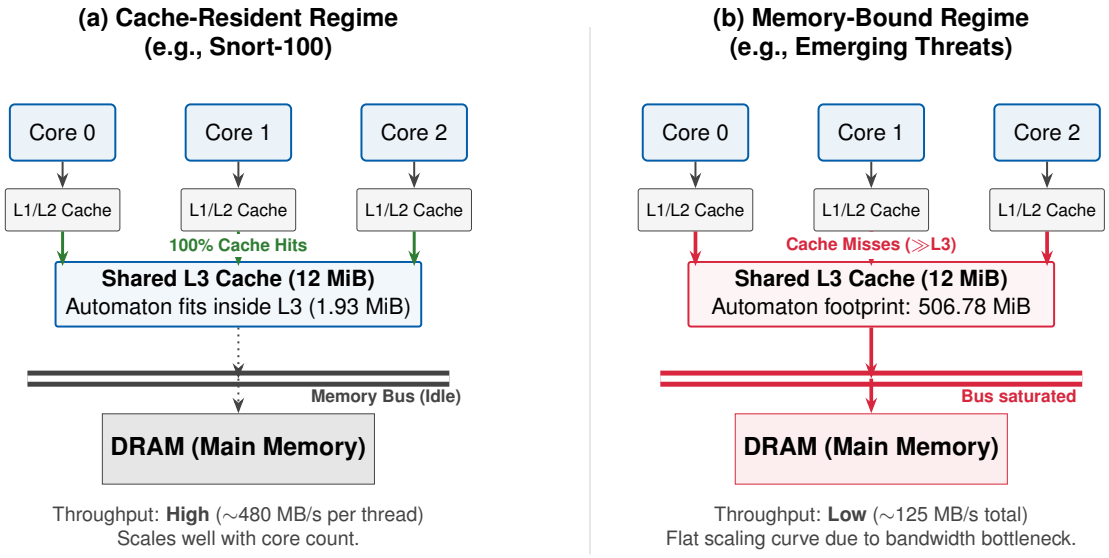
Source: Prepared by the author. Corpus fixed; single thread.

Throughput falls monotonically as the automaton grows, from 483.1 MB/s at 1.93 MiB (Snort-100) to 125.6 MB/s at 506.78 MiB (Emerging Threats)—a 74.0% reduction across an automaton roughly $260\times$ larger. Since the corpus is identical in every row, the degradation tracks the automaton size alone, and not the text content.

A plausible explanation for this behaviour is the memory hierarchy of the test platform. Each input byte triggers a lookup in the state-transition table; on the i5-1235U the largest dictionary produces an automaton about forty-two times the 12 MB shared last-level cache, so an increasing share of transitions is likely to miss cache and reach main memory as the automaton grows. This attribution is an inference: memory bandwidth, cache-hit and cache-miss counters were not instrumented, and the magnitude of the effect is specific to this platform's cache size rather than a property of the algorithm itself. Figure 11 illustrates the two regimes schematically.

The local conclusion is that, on this platform, the automaton footprint—not the text content—is the dominant factor in single-threaded throughput. The same degradation also persists under parallel execution: across the footprint sweep at twelve threads, the speedup of the topology-aware chunked strategy (against the sequential baseline) falls from $6.68\times$ for Snort-100 to $2.63\times$ for Emerging Threats, consistent with the memory subsystem rather than the thread count being the binding factor. This is revisited in the scaling analysis below, and it frames the interpretation of every subsequent result.

Figure 11: Cache-residency regimes governing throughput: (a) a small automaton fits within the shared L3, so transitions are likely to be served from cache; (b) a memory-bound automaton (≫L3) is likely to produce more cache misses and place sustained pressure on the memory bus, flattening the scaling curve.



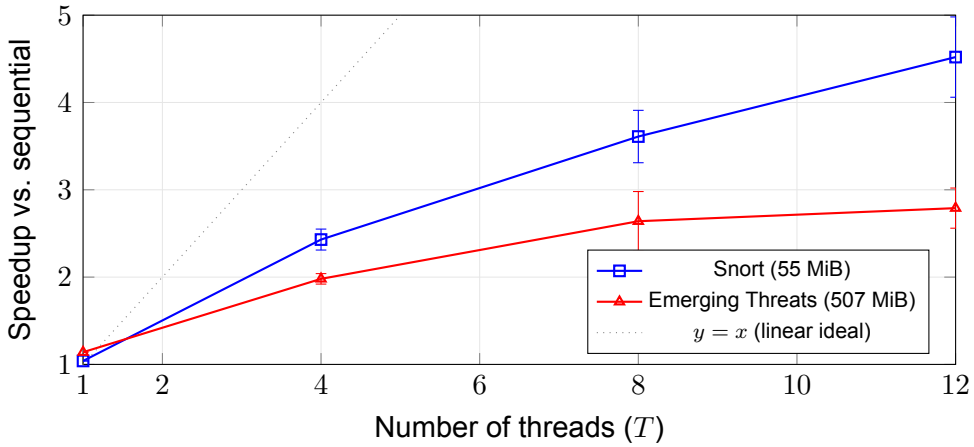
Source: Prepared by the author.

6.2 SCALABILITY OF TEXT PARALLELISM

This experiment evaluates the primary parallelization strategy—partitioning the input text across worker threads—by measuring how search speedup scales with the number of worker threads. It tests the hypothesis, stated in the methodology, that text chunking scales sublinearly on the hybrid processor, and it does so for the two footprint regimes identified above: a moderate dictionary only a few times larger than the cache (Snort, 55 MiB) and a memory-bound dictionary far larger than it (Emerging Threats, 507 MiB). The measurements use the largest corpus (∼10.6 GiB, `enron_x8`) to minimize scheduler and thermal noise.

The thread count is swept up to twelve. This is not an arbitrary cut-off: the i5-1235U exposes exactly twelve hardware threads (two Performance cores with simultaneous multithreading, plus eight Efficiency cores), so $T = 12$ is the full hardware concurrency of the platform and larger values would merely oversubscribe the cores. The sweep therefore covers the platform's entire thread budget; it does not claim to have located a universal threading ceiling for the algorithm, whose behaviour on processors with higher core counts is a matter of external validity addressed later. Figure 12 shows the resulting speedup curves.

Figure 12: Speedup of text parallelism versus thread count, for a moderate dictionary (Snort, 55 MiB; `pthread_chunked_v3_flat`) and a memory-bound dictionary (Emerging Threats, 507 MiB; `pthread_chunked_flat`), on the ∼10.6 GiB corpus (`enron_x8`). Error bars denote ± 1 standard deviation, derived from the per-run coefficient of variation over five timed iterations; they widen with thread count as scheduling and thermal noise grow.



Source: Prepared by the author.

The two regimes behave differently. With the moderate dictionary, the best text-parallel strategy rises steadily to $4.52\times$ at twelve threads ($1{,}087.1$ MB/s; CV 10%); on the canonical corpus the dynamic-dispatch strategy reaches $4.79\times$ ($1{,}148.1$ MB/s; CV 2%), the highest search speedup observed on the intrusion-detection-scale dictionaries. With the memory-bound dictionary the curve flattens far earlier: the best strategy reaches only $2.79\times$ at twelve threads (CV 8%), and on the canonical corpus the peak occurs near six threads ($2.85\times$; CV 16%) before declining at higher thread counts. Both regimes scale sublinearly.

The early plateau of the memory-bound regime is the central observation of this experiment. Its most likely cause is contention for the shared memory bus: once the automaton is about forty-two times the cache, adding threads—particularly the slower Efficiency cores beyond the two Performance cores—no longer raises throughput, which is consistent with the memory bus, rather than the core count, being the binding resource. This remains an inference drawn from the shape of the scaling curves and their agreement with the footprint experiment, not a direct measurement: memory bandwidth and cache counters were not instrumented, a limitation revisited in the discussion of threats to validity.

In the moderate-dictionary regime, where bandwidth is not the binding constraint, the $4.52\times$ ceiling at twelve threads is consistent with Amdahl's law [10]: the observed 37.7% parallel efficiency corresponds to an effective serial fraction of roughly 15%. On this hybrid platform that fraction is best read as an effective, observed quantity rather

than a fixed algorithmic constant, because it conflates genuine serialization (merge coordination and work dispatch), the P/E asymmetry of the pool—in which the four Performance-core hardware threads carry more per-thread weight than the eight Efficiency cores—and residual memory contention. Parallel efficiency (Eq. 4) makes the contrast explicit: at twelve threads the moderate regime achieves 37.7% efficiency ($4.52 \times /12$) against 23.3% ($2.79 \times /12$) for the memory-bound regime. That this gap mirrors the cache-residency difference far more than the per-thread synchronization cost is why it is attributed here primarily to memory behaviour rather than to threading overhead—again as an inference consistent with the footprint result, not a counter-level measurement.

The local conclusion is that text parallelism delivers a sublinear but substantial speedup whose ceiling is set by the footprint regime: useful and robust where the working set is moderate, and bounded by memory behaviour once the automaton is much larger than the cache. This confirms the sublinear-scaling hypothesis and sharpens it, by tying the ceiling to the automaton footprint rather than to the number of cores.

A separate experiment fixed the dictionary and strategy while changing the corpus, to confirm that the text content is not the main factor. The sequential throughput on the two corpora is nearly identical (241.0 MB/s versus 248.4 MB/s), and the twelve-thread speedup is of the same order on both ($4.42\times$ on the e-mail corpus versus $3.66\times$ on the encyclopedic corpus). The apparent gap, however, is not statistically distinguishable: the e-mail measurement carries a coefficient of variation of roughly 66%, so the two figures overlap within run-to-run variation and the difference should not be read as a genuine corpus effect. Because Aho-Corasick performs one transition per input byte regardless of match density, the footprint of the automaton dominates the behaviour far more than the characteristics of the corpus.

6.3 THE FLATTENED OUTPUT LAYOUT

The proposed output-layout optimization replaces the pointer-chasing emission of matches with a sequential read of a contiguous array. Because it changes only the memory layout, its effect can be measured in isolation on a single thread. Table 8 shows that, on the i5, the gain is regime-dependent and increases with memory pressure.

Table 8: Single-threaded gain of the flattened output layout, by regime.

Regime (dictionary)	Automaton	Baseline (MB/s)	Flattened (MB/s)	Gain
Snort-100 ($\ll$L3)	1.93 MiB	483.1	488.1	$+1.0\%$
Snort (full, $>$L3)	55.23 MiB	240.7	252.0	$+4.7\%$
Emerging Threats ($\gg$L3)	506.78 MiB	125.6	142.0	$+13.0\%$

Source: Prepared by the author. Single thread.

When the automaton fits in cache, the optimization provides almost no benefit ($+1.0\%$), because the linked structures are already cache-resident. When the automaton is far larger than the cache, the contiguous layout avoids the scattered misses of the pointer walk and delivers $+13.0\%$ on a single thread, which is the cheapest gain available in the memory-bound regime. In the parallel case, the layout is inherited by the chunked strategies, and the flattened variants lead or tie their pointer-chasing counterparts in both intrusion-detection-scale regimes; the best results — $4.52\times$ for the moderate dictionary at the multi-gigabyte scale and $2.85\times$ at the peak of the memory-bound regime on the canonical corpus — come from strategies that incorporate this layout. It is worth noting that the gain does not compound multiplicatively with text parallelism in the moderate regime: there, the DRAM stalls of the transition table already partially mask the emission cost, so the layout provides a smaller improvement than its single-threaded figure would suggest. The optimization is therefore best described as a cheap, broadly applicable improvement whose payoff is largest exactly where the problem is hardest. On the i5 this payoff increases with the automaton footprint; the precise progression of the gain is platform-dependent, so the qualitative pattern—a larger benefit under heavier memory pressure—is more robust than the specific per-regime percentages.

6.4 LOAD BALANCING ON HETEROGENEOUS CORES

This experiment asks whether the balancing policies proposed for the hybrid processor improve mean throughput or instead reduce run-to-run instability. It uses the canonical Snort workload at twelve threads and compares both throughput and the coefficient of variation of the per-iteration timing. The hypothesis is that uniformly sized text segments can create stragglers on the i5: threads assigned to Efficiency cores may finish later than those on the faster Performance cores, leaving early finishers idle at the barrier. The three partitioning policies are defined in the proposal chapter (Figure 7). Table 9 reports, for the canonical workload at twelve threads, the coefficient of variation of the per-iteration timing.

Table 9: Run-to-run stability of the text-parallel strategies (twelve threads).

Strategy	Throughput (MB/s)	Variation (CV)
Static homogeneous chunking	1,064.7	39.9%
Static chunking (cache-padded)	902.0	52.9%
Topology-aware (frequency-weighted)	879.8	1.7%
Dynamic dispatch	864.4	3.3%
Flattened output with chunking	1,007.2	0.4%
Topology-aware with flattened output	994.2	4.3%

Source: Prepared by the author. Canonical workload, twelve threads; dedicated stability diagnostic (three timed iterations under sustained load). The mean throughputs are subject to the run-to-run variance the table quantifies and are not directly comparable with the sustained figures of the scalability analysis.

The static homogeneous strategies exhibit a coefficient of variation between 39.9% and 52.9%, consistent with slower cores intermittently delaying the overall execution. Both solutions proposed in the design reduce this variation: frequency-weighted partitioning reduces it to 1.7% and dynamic dispatch to 3.3%, while the flattened chunking strategy is the most stable of all (down to 0.4%) — indicating that uniform partitioning makes the run-to-run timing sensitive to how the scheduler places the slow-core segments, rather than imposing a fixed penalty. This shows that topology awareness does not make the transition function faster; it removes idle time at the barrier, trading a small change in mean throughput for a large gain in predictability. The improvement is in predictability and worst-case latency, not in mean throughput, and it does not by itself change the ranking of strategies, because the underlying ceiling remains the memory subsystem. Because the stragglers arise from the Performance/Efficiency split of this processor, this is a result specific to hybrid CPUs such as the i5: on a homogeneous machine, uniformly sized segments would not produce the same imbalance, and topology-aware sizing would have little to correct. The value demonstrated here is therefore the decoupling of timing predictability from mean throughput on heterogeneous cores, rather than a universal throughput gain.

6.5 PARALLEL CONSTRUCTION OF THE AUTOMATON

This experiment evaluates whether parallel construction reduces operational rule-reloading latency. The implemented variant parallelizes the breadth-first traversal that computes failure links, because each level depends only on shallower levels and can therefore be processed in parallel with a barrier between levels. The trie insertion remains sequential; only the traversal is parallelized. The level-synchronous construction strategy and its dependency structure are defined in the proposal chapter (Figure 9).

Table 10 reports the build speedup across thread counts, rather than a single peak, so that the full thread-count behaviour is visible.

Table 10: Parallel automaton construction: build speedup versus the sequential build, by thread count.

Dictionary	States	Seq. (ms)	$T{=}2$	$T{=}4$	$T{=}8$	$T{=}12$
Snort-100	1,939	1.5	$0.80\times$	$0.83\times$	$0.71\times$	$0.60\times$
Snort (full)	55,479	48.2	$1.00\times$	$1.19\times$	$1.17\times$	$0.91\times$
Emerging Threats	508,896	485.2	$1.18\times$	$1.43\times$	$1.55\times$	$1.41\times$

Source: Prepared by the author. Build speedup is the sequential build time divided by the parallel build time at each thread count; values below $1.0\times$ mean the parallel build is slower than the sequential one.

The thread-count breakdown shows that parallel construction only pays off for the large dictionary, where it peaks at $1.55\times$ at eight threads (reducing the build from 485 to roughly 313 milliseconds) before easing to $1.41\times$ at twelve. The moderate dictionary barely benefits, peaking at $1.19\times$ at four threads and regressing to $0.91\times$ at twelve, while the smallest dictionary is slower than sequential at every thread count, because the per-level spawn-and-join overhead exceeds the work. This optimization should therefore be understood as an operational benefit—a reduction in rule-reloading latency for systems that frequently update their signatures—rather than as a contribution to scanning throughput; on the multi-gigabyte workloads it accounts for less than 1% of the total time.

6.6 PARTITIONING THE DICTIONARY: A QUALIFIED NEGATIVE RESULT

Partitioning the dictionary, rather than the text, was evaluated as an alternative, assigning patterns to shards by their first byte (prefix sharding). Table 11 reports the throughput across shard counts and compares the result against plain text parallelism on the canonical workload.

Table 11: Dictionary sharding (by prefix) across shard counts, compared to text parallelism (Snort, canonical corpus).

Configuration	Throughput (MB/s)	vs. sequential
Prefix sharding, 2 shards	322.6	$1.35\times$
Prefix sharding, 4 shards	319.9	$1.34\times$
Prefix sharding, 6 shards (peak)	327.2	$1.37\times$
Prefix sharding, 8 shards	291.3	$1.22\times$
Prefix sharding, 12 shards	276.6	$1.15\times$
Text chunking, flat (12 threads)	976.1	$4.07\times$

Source: Prepared by the author. Speedup is relative to the single-threaded sequential baseline (239.6 MB/s); text chunking is shown at twelve threads for reference.

Prefix sharding peaks at $1.37\times$ over the baseline at six shards, but it is already within 2% of that peak at just two shards ($1.35\times$) and declines past six. This early saturation indicates that the benefit comes from improved cache locality within each smaller sub-automaton rather than from added parallelism. Even at its best it attains only about a third ($0.34\times$) of the throughput of plain text parallelism. The conclusion is that, on this class of hardware, partitioning the text dominates partitioning the dictionary: a real but small effect, not a competitive strategy.

6.7 COMBINED STRATEGY EVALUATION: SHARDING WITH CHUNKING

A natural hypothesis is that combining dictionary sharding (for cache locality) with text chunking (for parallelism) would recover the throughput lost in the memory-bound regime, by making each sub-automaton small enough to approach the cache. Table 12 tests this two-dimensional composition against flat-output text chunking and disproves the hypothesis.

Table 12: Two-dimensional composition versus flat-output text chunking (twelve threads).

Workload	Composition (MB/s)	Text chunking (MB/s)	Ratio
Snort, canonical corpus	785.4	976.1	$0.80\times$
Snort, $\sim$10.6 GiB	861.6	1,073.3	$0.80\times$
Emerging Threats, canonical corpus	289.0	312.3	$0.93\times$
Emerging Threats, $\sim$10.6 GiB	314.5	350.0	$0.90\times$

Source: Prepared by the author. Twelve threads in both columns; the comparator is the flat-output text chunking of Table 11.

The composition is slower than flat-output text chunking in every measured scenario, by 7% to 20%. This is consistent with duplicated text traffic becoming the limiting resource: each shard must scan the entire input text, so K shards multiply the text traffic on the memory bus by a factor of K. In the memory-bound regime, the extra reads plausibly outweigh any locality gained inside the smaller sub-automata. To shrink the sub-automaton below the last-level cache for the largest dictionary would require more than forty shards, multiplying the text traffic proportionally, which is unlikely to be recoverable on a memory bus of this bandwidth. This negative result is an important contribution: it closes a plausible region of the design space and reinforces the conclusion that text parallelism is the correct axis on memory-bound workloads. This difference in memory traffic is illustrated in the proposal chapter (Figure 10).

6.8 END-TO-END CONSISTENCY CHECK

The preceding sections measured construction and search in isolation. This final check verifies that those results hold for the complete pipeline—automaton construction followed by a single full scan of the $\sim$10.6 GiB corpus, reported as wall-clock time. It introduces no new experiment: to keep the figures auditable, each cell is reconstructed from the same canonical sweep as the measured build time plus one full-corpus search pass (Table 13).

Table 13: End-to-end wall-clock on the ∼10.6 GiB corpus, as construction time plus a single full-corpus scan.

Dictionary	Sequential (ms)	Best parallel (ms)	Speedup
Snort (full)	45,174	10,027	$4.50\times$
Emerging Threats	86,998	31,517	$2.76\times$

Source: Prepared by the author, derived from the 2026 canonical sweep. Each cell is the measured build time plus one full-corpus search pass (*build_ms* + *mean_ms*); the parallel column uses the best twelve-thread strategy with a sequential build.

The end-to-end speedups are $4.50\times$ for the moderate dictionary (roughly forty-five seconds down to ten) and $2.76\times$ for the memory-bound one (87.0 to 31.5 seconds, with flat-output text chunking and a sequential build). Because construction accounts for under one percent of the pipeline—49 ms of $45{,}174$ ms for Snort and 541 ms of $86{,}998$ ms for Emerging Threats—these figures track the search-only speedups of Figure 12 almost exactly ($4.50\times$ versus $4.52\times$; $2.76\times$ versus $2.79\times$). The pipeline is therefore dominated by the scanning phase, and the end-to-end measurement adds no scaling beyond what the search analysis already establishes; it serves only to confirm that the earlier speedups hold end to end.

6.9 PORTABILITY ON A HOMOGENEOUS WORKSTATION

All the figures reported so far were obtained on a single hybrid mobile processor. To test whether the conclusions hold beyond that platform, the full sweep was replayed on a homogeneous desktop processor: an AMD Ryzen 9 9950X with sixteen identical cores (thirty-two hardware threads), DDR5 memory, and a last-level cache organized as two 32 MiB slices, one per core complex. This portability run does not replace the canonical study; the i5-1235U sweep remains the source of truth for the tables and figures of this chapter. Its purpose is narrower: to separate the findings that follow from the algorithm and the memory-bound regime from those that follow from the specific hybrid silicon of the reference machine. Table 14 summarizes the comparison.

Table 14: Portability comparison between the hybrid reference laptop and the homogeneous workstation. The i5-1235U remains the canonical platform; the Ryzen 9 9950X run probes external validity.

Quantity	**i5-1235U (12T)**	**Ryzen 9 9950X (32T)**
Peak speedup, Snort (55 MiB)	$4.52\times$ (T=12)	$22.72\times$ (T=32)
Peak speedup, Emerging Threats (507 MiB)	$2.79\times$ (T=12)	$19.79\times$ (T=32)
Peak throughput, Snort	$1{,}087$ MB/s	$7{,}468$ MB/s
Peak throughput, Emerging Threats	350 MB/s	$4{,}186$ MB/s
Single-thread cache cliff	-74.0%	-67%
Parallel build ceiling (ET, eight threads)	$1.55\times$	$1.57\times$
Chained-chunking run-to-run CV	39.9–52.9%	0.10%
Winning search strategy	Snort: topology-aware flat; ET: flat	dynamic flat

Source: Prepared by the author, from the canonical i5 sweep (2026) and the Ryzen 9 9950X portability run. The i5 figures are those reported earlier in this chapter; the workstation figures come from a sweep on the same corpora and dictionaries.

The workstation **confirms** the central findings of the study. The footprint–throughput coupling persists: single-threaded throughput still collapses as the automaton outgrows the cache — by 67% on the workstation against 74.0% on the i5 — so the cache cliff is a property of the workload rather than of one particular cache size. Partitioning the text again dominates partitioning the dictionary, the flattened emission layout again helps, and parallel construction again saturates at an algorithmic ceiling rather than a hardware one: $1.57\times$ at eight threads for the largest dictionary, almost identical to the $1.55\times$ measured on the i5 despite the wider homogeneous core pool. The bottleneck, the winning axis, and the cheap layout optimization therefore carry across microarchitectures.

The workstation **contests** readings that the single-machine data made tempting. The early plateau is not universal: where the i5 saturates at twelve threads on the multi-gigabyte corpus and, on the canonical corpus, the memory-bound curve even regresses

past six, the workstation scales monotonically to $22.72\times$ at thirty-two threads for the moderate dictionary ($7{,}468$ MB/s) and to $19.79\times$ for the memory-bound one ($4{,}186$ MB/s). The earlier observation that more threads become counterproductive is thus specific to the constrained laptop platform and is not a general property of the algorithm. By the same token, the inherent serial fraction inferred from the i5 (around 15%) overstates the algorithm's coordination cost: with uniform cores, parallel efficiency stays high well beyond the i5's saturation point, which indicates that much of the apparent serial fraction was the Performance/Efficiency asymmetry of the hybrid pool rather than serial code. The flattened layout, finally, does not keep growing without bound; its single-thread gain settles near $+4\%$ on the workstation instead of reaching the $+13.0\%$ peak observed under the i5's weaker memory subsystem.

The workstation also **reveals** which contributions are intrinsically tied to hybrid hardware. The load-balancing machinery — frequency-weighted, topology-aware chunking — exists to tame the stragglers that heterogeneous cores produce: on the i5, static chained chunking varies by 39.9 to 52.9% from run to run, whereas on the uniform workstation the same strategy is essentially deterministic (0.10%). With identical cores there are no stragglers to correct, and the design space collapses onto a single winner, dynamic dispatch with the flattened layout (`pthread_dynamic_flat`), which leads in both regimes. The finding that no single strategy is best across regimes is therefore a consequence of heterogeneity, not a permanent feature of the problem.

In summary, the workstation does not overturn the study; it widens its external validity. The memory-bound regime, the primacy of text partitioning, and the value of a cache-friendly layout hold on both machines, while the saturation point, the topology-aware balancing, and the strategy ranking are recognized as platform-dependent effects of the hybrid reference machine. The i5-1235U sweep remains the canonical basis of this chapter; the workstation establishes how far its conclusions can be carried.

6.10 DISCUSSION AND POSITIONING

Taken together, the i5 results converge on a consistent reading. On this platform, the dominant obstacle to accelerating Aho-Corasick at intrusion-detection scale appears to be microarchitectural: once the automaton greatly exceeds the last-level cache, performance is bounded by the memory subsystem, and the most effective response is to parallelize the text while keeping the per-byte transition cost as low as possible. No single strategy is best across all regimes; the fastest strategy switches between dynamic dispatch (moderate dictionary on the canonical corpus) and the flattened variants, with and without topology awareness, in the remaining scenarios. This pattern is consistent with the main bottleneck shifting, as the dictionary grows, from thread scheduling toward memory bandwidth—the regime in which a cache-friendly emission path helps

the most—an interpretation inferred from the measurements rather than confirmed by hardware-counter instrumentation. This conclusion is not an artifact of the reference laptop: as the portability subsection above shows, the memory-bound bottleneck, the primacy of text partitioning, and the value of the flattened layout reappear on a homogeneous sixteen-core workstation, even though that machine scales far further before saturating.

These findings can be positioned against the related literature. Compared to approaches that restructure the automaton to improve cache friendliness, such as the head–body decomposition of the matcher for multi-core intrusion detection [2], the present work attains a comparable order of magnitude of throughput while leaving the classical automaton mathematically intact, which keeps correctness easy to validate against the sequential reference. The failure-less variant of the algorithm designed for massively parallel accelerators [5] is, however, unsuitable for a multi-core CPU in this setting, because its independent per-byte traversals multiply work in a way that the memory bus is unlikely to absorb; this is consistent with the negative result observed here for over-partitioning. Single-threaded vectorized engines that replace the automaton with SIMD literal matching [25] operate on an orthogonal axis, exploiting data parallelism within a thread rather than across threads, and are therefore complementary to, rather than competitive with, the inter-thread parallelism studied here. The finding that layout-level changes to the emission path yield real gains aligns with work on transition-table compression for the same algorithm [24]. Finally, the use of realistic rule sets and large corpora follows the guidance on representative signature-based intrusion-detection benchmarks [6], which is what allows the memory-bound regime to be studied, rather than an artificially cache-friendly one.

7

7 CONCLUSION

This thesis investigated data-parallel scanning strategies and microarchitectural optimizations to accelerate the Aho-Corasick multi-pattern matching algorithm on shared-memory multi-core CPUs, using the POSIX Threads programming model. The work targeted a specific computational regime rather than a particular machine: the behaviour of multi-pattern matching when the automaton greatly exceeds the last-level cache, which is typical for signature-based intrusion detection with realistic rule sets.

At the level of computational regime, three findings are supported by the platforms tested and form the core contribution. First, once the automaton no longer fits in the last-level cache, the dominant constraint is the memory subsystem, not the matching logic: automaton footprint governs throughput in the large-corpus workloads. The broader i5 sweep also indicates that corpus content is secondary in this regime, but that cross-corpus axis was not repeated on the workstation. Second, the axis with the most leverage is text-level partitioning with a read-only shared automaton; partitioning the dictionary instead of the text yields only a small, locality-driven gain that saturates early and never approaches text parallelism. Third, a flat output layout that replaces pointer-chasing with contiguous reads is the simplest effective optimization and remains useful under memory pressure. Two strategies were falsified on every platform: dictionary sharding alone never approaches text parallelism, and combining sharding with chunking is consistently slower than flat-output text chunking, because scanning the text once per shard multiplies the memory traffic that is already the bottleneck. These negative results are part of the contribution: they rule out plausible approaches and reinforce that, whenever the working set is memory-bound, DRAM bandwidth—not the matching code—is the binding constraint, so text parallelism combined with a cache-friendly layout is the correct approach. What does *not* generalize is the absolute magnitude of the gains and the shape of the scaling curve: those depend on the memory subsystem and core topology of the machine, as the two platforms below make explicit.

7.1 FINDINGS ON THE REFERENCE PLATFORM

The complete experimental study was conducted on the Intel i5-1235U, a mobile hybrid processor with two performance cores, eight efficiency cores, and a 12 MiB last-level cache; it remains the source of truth for the headline numbers, tables, and

figures of this thesis. Text-level partitioning with a boundary overlap of $L_{\max} - 1$ bytes scales sublinearly, reaching $4.52\times$ at twelve threads on the multi-gigabyte corpus and a canonical-corpus peak of $4.79\times$ via dynamic dispatch. As the dictionary grows from a cache-resident set to one forty-two times larger than the last-level cache, single-threaded throughput falls by 74.0%, and the parallel speedup drops accordingly: in the memory-bound regime the speedup peaks at $2.85\times$ near six threads on the canonical corpus and then regresses as further threads are added. This regression is a property of this constrained platform, not of the algorithm: the i5's narrow memory subsystem and its two performance cores saturate before all twelve hardware threads are used, an effect that the homogeneous workstation in the next section does not reproduce. The corresponding parallel efficiency at twelve threads is 37.7% for the moderate-dictionary regime and 23.3% for the memory-bound one, reflecting DRAM saturation as the dominant overhead rather than an inherent serial fraction of the algorithm.

On this heterogeneous processor, frequency-weighted partitioning and dynamic dispatch removed the load imbalance of uniform segments, reducing the per-thread coefficient of variation from up to 53% to under 5%. This improves predictability rather than peak throughput, since memory remains the bottleneck. The proposed flat output layout reached $+13.0$% on a single thread exactly in the memory-bound regime where it matters most. Parallelizing the construction of the automaton accelerated the build of the largest dictionary by up to $1.55\times$, a useful operational gain for rule reloading, though negligible in the total time of a multi-gigabyte scan. End-to-end, the full pipeline achieves $4.50\times$ on the moderate dictionary and $2.76\times$ on the memory-bound one—figures that track the search-only speedups, since construction is under one percent of the runtime.

The three specific objectives from the introduction were fully met on this platform. The first—to design and implement a parallel Aho-Corasick algorithm in C using the POSIX Threads API, exploiting intra-input parallelism on shared-memory multi-core CPUs—was met through a family of text-partitioning, load-balancing, and output-layout strategies whose correctness was validated exhaustively across all tested workloads. The second—to quantify performance gains in terms of speedup, throughput, and parallel efficiency against a sequential baseline—was met by reporting search speedups of $4.79\times$ (canonical corpus, dynamic dispatch) and $4.52\times$ (multi-gigabyte corpus, flat-output chunking), alongside parallel efficiencies of 37.7% and 23.3% for the moderate-dictionary and memory-bound regimes, respectively. The third—to analyze scalability under different workloads, pattern sets, and thread counts—was met through a systematic test across seven thread counts, three corpora, and automata covering a forty-two-fold range of cache capacity, which identified and quantified the memory bandwidth saturation point and the subsequent performance drop.

7.2 PORTABILITY TO A HOMOGENEOUS MANY-CORE CPU

A portability run on a homogeneous AMD Ryzen 9 9950X (sixteen identical cores, thirty-two hardware threads, a 64 MiB last-level cache split across two core complexes), executed on the same corpora and dictionaries, separates the general findings above from the artifacts of the hybrid mobile chip. It confirms the core thesis. The cache cliff persists: single-threaded throughput still falls by about 67% from the smallest to the largest automaton, so the memory-bound regime is structural rather than an artifact of the i5's small cache. The strategy hierarchy is unchanged—text partitioning beats pattern sharding, the flat layout beats the chained one, and parallel construction tops out at $1.57\times$, essentially the $1.55\times$ algorithmic ceiling measured on the i5 despite eight times as many cores.

At the same time, the workstation shows that several phenomena reported for the i5 are specific to constrained or heterogeneous hardware, not universal laws. Where the i5 regresses past six threads, the Ryzen scales monotonically to $22.72\times$ on the moderate dictionary and $19.79\times$ on the memory-bound one at thirty-two threads, sustaining about 89% parallel efficiency at sixteen threads. The low i5 efficiency therefore reflects a narrow memory subsystem compounded by performance/efficiency-core asymmetry, not a large serial fraction intrinsic to the algorithm. The elaborate topology-aware load balancing that was central on the i5 becomes unnecessary on identical cores, where simple static distribution already drives the per-thread coefficient of variation to about 0.1%; and the finding that no single strategy dominates across regimes is an i5 effect—on the uniform machine a single flat dynamic variant wins in both regimes. A larger machine with uniform cores and more memory channels thus delays the bottleneck and scales much further, but it does not eliminate the bottleneck: the memory-bound regime survives the change of hardware.

7.3 THREATS TO VALIDITY

The principal study rests on a single mobile, hybrid processor with a limited sustained-load envelope, which made long tests sensitive to scheduler, affinity, and operating-condition variance; the reported numbers are conservative under-load values, and the variability is documented rather than hidden. The Ryzen 9 9950X run reduces, but does not eliminate, the external-validity threat: it confirms the memory-bound thesis on a second microarchitecture, yet both machines share a single-socket, single-NUMA-node design, and neither was instrumented with hardware performance counters. The microarchitectural explanations are therefore consistent with the timing data but were not confirmed by direct telemetry of cache misses, memory bandwidth, or pipeline stalls. A controlled re-run of the i5 sweep on a colder, headless machine reproduced the sequential baseline within about 3% and the overall shape of the scaling curves, which supports reproducibility; however, the saturated twelve-thread peaks varied by tens

of percent between separate invocations of the same configuration, an inter-invocation variance that the intra-run coefficient of variation does not capture. The single-run maxima at the saturation point should thus be read as indicative rather than exact.

7.4 FUTURE WORK

Four directions follow from these limitations. The first is to add hardware performance counters to the benchmarks, measuring cache misses, memory bandwidth, and pipeline stalls directly, so that the memory bottleneck is confirmed with direct measurements instead of inferred from execution times. The second, motivated by the re-run above, is to report the saturated i5 operating points as medians over several independent invocations rather than as the maximum of a single sweep, so that the inter-invocation variance is averaged out rather than amplified. The third is to settle the dynamic-granularity question—the trade-off between task size and scheduling overhead—with several repetitions per granularity setting, ideally over an unevenly loaded corpus, since a single invocation per setting is dominated by the same operating-regime variance. The fourth is to widen the evaluation across more architectures and corpora—additional microarchitectures, larger NUMA topologies, and document collections with different content statistics—building on the two-platform evidence already gathered to map where strategies such as dictionary sharding might finally become beneficial.

REFERENCES

References

[1] P. Pacheco, *An Introduction to Parallel Programming*, 2nd ed. Morgan Kaufmann, 2011.

[2] C.-L. Lee and T.-H. Yang, “A flexible pattern-matching algorithm for network intrusion detection systems using multi-core processors,” *Algorithms*, vol. 10, no. 2, p. 58, 2017.

[3] VirusTotal, “YARA: The pattern matching swiss knife,” 2025. [Online]. Available: https://virustotal.github.io/yara/

[4] A. V. Aho and M. J. Corasick, “Efficient string matching: An aid to bibliographic search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–343, 1975.

[5] V. Thambawita, R. G. Ragel, and D. Elkaduwe, “An optimized Parallel Failure-less Aho-Corasick algorithm for DNA sequence matching,” *CoRR*, vol. abs/1811.10498, 2018.

[6] M. Aldwairi, M. A. Alshboul, and A. Seyam, “Characterizing realistic signature-based intrusion detection benchmarks,” in *Proceedings of the 6th International Conference on Information Technology (ICIT 2018)*. Association for Computing Machinery, 2018.

[7] E. Sitaridi, O. Polychroniou, and K. A. Ross, “SIMD-accelerated regular expression matching,” in *Proceedings of the 12th International Workshop on Data Management on New Hardware (DaMoN ’16)*. Association for Computing Machinery, 2016.

[8] D. R. Butenhof, *Programming with POSIX Threads*. Addison-Wesley, 1997.

[9] M. J. Flynn, “Very high-speed computing systems,” *Proceedings of the IEEE*, vol. 54, no. 12, pp. 1901–1909, 1966.

[10] A. Grama, A. Gupta, G. Karypis, and V. Kumar, *Introduction to Parallel Computing*, 2nd ed. Addison-Wesley, 2003.

[11] Intel Corporation, “Intel Core i5-1235U Processor (12M Cache, up to 4.40 GHz) – Product Specifications,” Intel ARK, 2022. [Online]. Available: https://ark.intel.com/content/www/us/en/ark/products/226266/

[12] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS ’67)*, 1967, pp. 483–485.

[13] J. L. Gustafson, “Reevaluating Amdahl’s law,” *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.

[14] E. Fredkin, “Trie memory,” *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, 1960.

[15] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.

[16] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley Professional, 2011.

[17] D. Gusfield, *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997.

[18] D. E. Knuth, J. H. Morris Jr., and V. R. Pratt, “Fast pattern matching in strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.

[19] R. S. Boyer and J. S. Moore, “A fast string searching algorithm,” *Communications of the ACM*, vol. 20, no. 10, pp. 762–772, 1977.

[20] J. Qu, G. Zhang, Z. Fang, and J. Liu, “A parallel algorithm of string matching based on Message Passing Interface for multicore processors,” *International Journal of Hybrid Information Technology*, vol. 9, no. 3, pp. 31–38, 2016.

[21] D. Deyannis, E. Papadogiannaki, G. Chrysos, K. Georgopoulos, and S. Ioannidis, “The diversification and enhancement of an IDS scheme for the cybersecurity needs of modern supply chains,” *Electronics*, vol. 11, no. 13, p. 1944, 2022.

[22] T. Jepsen, D. Alvarez, N. Foster, C. Kim, J. Lee, M. Moshref, and R. Soulé, “Fast string searching on PISA,” in *Proceedings of the 2019 ACM Symposium on SDN Research (SOSR ’19)*. Association for Computing Machinery, 2019.

[23] D. Tovarňák, “Practical multi-pattern matching approach for fast and scalable log abstraction,” in *Proceedings of the 11th International Joint Conference on Software Technologies (ICSOFT-EA 2016)*. SCITEPRESS, 2016, pp. 319–329.

[24] D. Regéciová, D. Kolář, and M. Milkovič, “Pattern matching in YARA: Improved Aho-Corasick algorithm,” *IEEE Access*, vol. 9, pp. 62 857–62 866, 2021.

[25] H. Xu, H. Chang, W. Zhu, Y. Hong, G. Langdale, K. Qiu, and J. Zhao, “Harry: A scalable SIMD-based multi-literal pattern matching engine for deep packet inspection,” in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*. IEEE, 2023.

[26] B. Nichols, J. Buttlar, and J. P. Farrell, *Pthreads Programming: A POSIX Standard for Better Multiprocessing*. O'Reilly & Associates, 1996.

APPENDICES

Appendix A - Database Query Protocol

This appendix documents the protocol used to query the raw measurement database from which every result reported in this thesis is derived, including the end-to-end wall-clock times of Table 13. Those end-to-end figures are not a separate campaign: each is the sum of two quantities that the canonical sweep already records for the corresponding execution—the construction time (`build_ms`) and a single full-corpus search pass (`mean_ms`). They can therefore be reproduced and audited from the same database as every other figure. The benchmark campaign stores its measurements in a single typed SQLite database located at `parallel-aho-corasick/runs/i5/sweep.db`, containing one `runs` table (one row per timed execution) and a set of aggregating views. The headline figures are retrieved from these views instead of the raw execution logs.

9.1 ARTIFACT AVAILABILITY

The complete artifacts of this work are publicly available in two Git repositories: the LaTeX source of this document at `https://github.com/MTheus22/acceleration-of-pattern-matching-algorithms-using-parallel-programming`, and the C implementation—together with the searchers, the benchmark harness, the dataset descriptors, and the measurement database—at `https://github.com/MTheus22/parallel-aho-corasick`. Every empirical figure reported in this thesis is derived from the latter repository. After cloning it, the sequential and parallel searchers can be built and exercised through the project Makefile: `make` compiles the laboratory, `make test` checks that every strategy reproduces the exact match set of the sequential baseline, `make digest` runs the MD5 multiset validation described in Section 5.7 (up to sixty-four threads), and `make bench` executes the measurement sweep. The remainder of this appendix documents the schema and the canonical queries of the resulting measurement database, so that each headline number can be traced back to the raw measurements.

9.2 DATABASE SCHEMA

Each row of the `runs` table identifies an execution by its phase, pattern dictionary, corpus, searcher strategy, and thread count. It records the timing statistics (minimum, mean, median, and maximum wall-clock time in milliseconds, along with the coefficient of variation across the timed iterations), the derived throughput (in MB/s and Gbit/s), the input volume and match count, and the automaton descriptors (pattern count, number of states, footprint in kibibytes, and sequential or parallel build time). The views listed in Table 15 pre-aggregate the data so no headline figure requires ad hoc post-processing of the raw rows.

Table 15: Aggregating views of the measurement database.

View	Question it answers
`v_speedup`	Speedup of each searcher against the single-threaded sequential baseline.
`v_self_speedup`	Self-relative scalability: throughput at T threads over throughput at one thread.
`v_footprint`	Throughput as a function of the automaton footprint, at one and twelve threads.
`v_build`	Speedup of the parallel automaton construction over the sequential build.
`v_best`	Best-performing searcher for each dictionary, corpus and thread count.
`v_correctness`	Number of distinct match counts per dictionary and corpus (a value of one indicates agreement).

Source: Prepared by the author.

9.3 CANONICAL QUERIES

The queries below reproduce the main results of the study. They can be executed against the database using the standard `sqlite3` client.

```
-- Speedup curve of a searcher (Snort dictionary, canonical corpus)
SELECT thr, speedup_vs_seq FROM v_speedup
WHERE patterns='patterns_snort' AND corpus='enron_corpus'
  AND searcher='pthread_dynamic' ORDER BY thr;

-- Peak speedup per scenario (winning searcher and thread count)
SELECT patterns, corpus, searcher, thr, speedup_vs_seq FROM v_speedup w
WHERE speedup_vs_seq = (SELECT MAX(speedup_vs_seq) FROM v_speedup x
  WHERE x.patterns = w.patterns AND x.corpus = w.corpus)
ORDER BY patterns, corpus;

-- Footprint versus throughput at twelve threads (cache blowout)
SELECT patterns, automaton_mib, searcher, mbps FROM v_footprint
WHERE thr = 12 ORDER BY automaton_mib;

-- Speedup of the parallel automaton construction
SELECT patterns, build_threads, build_speedup FROM v_build
```

```
ORDER BY patterns, build_threads;

-- End-to-end wall-clock (build + one full-corpus scan) on enron_x8
SELECT patterns, searcher, thr, build_ms, mean_ms,
       build_ms + mean_ms AS e2e_ms
FROM runs
WHERE corpus='enron_x8' AND thr IN (0, 12)
  AND searcher IN ('sequential','pthread_chunked_flat','pthread_chunked_v3_flat')
ORDER BY patterns, searcher, thr;
```

Before any timing was reported, the correctness of every parallel strategy was verified using the `v_correctness` view. This view returns a single distinct match count for each dictionary and corpus, confirming that all strategies reproduce the exact match set of the sequential baseline.

idp

Ensino que
te **conecta**